

AI와 금융 필드에서 전천후로 활동하는 강지호입니다.

사회/경제 도메인 이해를 기반으로 LLM 엔지니어링을 결합해, 기획부터 개발·배포·운영까지
1인 체제로 SaaS 제품을 만들고 운영합니다.

EXPERIENCE 경력

- | | | |
|----|--|-------------------|
| 01 | StockHedge — 대표 · 1인 스타트업 | 2025.08 — 현재 |
| | AI 웹빌더 SaaS(aiwebbuilder.kr) 기획·개발·배포·운영 단독 수행 | |
| 02 | 민트투자자문 — AI Transformation | 2026.04 — 현재 |
| | 인턴 2026.04 — 2026.07 · 프리랜서 2026.07 — 현재
사내 통합 플랫폼 엔드투엔드 설계 및 LLM 엔지니어링 | |
| 03 | 토스페이 — Onboarding Operational Assistant | 2025.06 — 2025.08 |
| | 가맹점 온보딩 운영 프로세스 지원 및 BD 업무 | |

SELECTED WORK 주요 프로젝트

- | | | |
|----|--|-------------------|
| 01 | AIWebbuilder — 웹빌딩 플랫폼 | 2026.05 — 현재 |
| | 자연어 입력 기반 웹사이트 자동 생성 SaaS · 생성 품질/비용 최적화 | |
| 02 | 광고1동 상권 활성화 프로젝트 — 기획 / 총괄 | 2025.09 — 2025.12 |
| | 상권 데이터 분석 기반 실행안 수립 및 이해관계자 조율 | |
| 03 | 2025 수원 대학생 청년 정책 포럼 — 기획 / 총괄 | 2025.11 |
| | 포럼 전체 기획, 연사 섭외, 당일 운영 총괄 | |

ACTIVITIES & AWARDS 대외활동 · 공모전

- | | | |
|----|--------------------------------------|-------------------|
| 01 | 경기대학교 금융경제동아리 KFI — 초대 회장 | 2025.08 — 2025.12 |
| 02 | 제10회 DB GAPS 투자대회 | 2025.08 — 2025.12 |
| 03 | TV조선 서포터즈 | 2025.02 — 2025.04 |
| 04 | 2024 한국지능정보사회진흥원 데이터 크리에이터 대회 | 2024.09 — 2024.11 |
| 05 | 2024 한국발명진흥회 캠퍼스 특허 유니버시아드 | 2024.03 — 2024.07 |

REFERENCE (제목 클릭하시면 해당 링크로 넘어갑니다!) · 출판작 · 포트폴리오

- | | | |
|----|-------------------------------------|------------------|
| 01 | DB보험금융공모전 — VIX 레짐 원자재 선물 전략 | docs.google.com |
| 02 | 제10회 DB GAPS 투자대회 — 리서치 보고서 | docs.google.com |
| 03 | 금융경제분석 기말 연구 보고서 | drive.google.com |
| 04 | AIWebbuilder — AI 웹빌딩 플랫폼 | aiwebbuilder.kr |

강지호

KANG JI HO

姜知昊

CONTACT

BORN
2001.09.11

PHONE
010-7666-4510

EMAIL
official@aiwebbuilder.kr

WEB
github.com/StockHedge/portfolio

BASE
경기도 화성시 동탄구 메타폴리스

GITHUB
https://github.com/StockHedge

EDUCATION

경기대학교 경제학과

경제학 학사 · 졸업

2022.03 — 2026.02

FOCUS

AI Product

LLM Engineering

SaaS 운영

금융 · 퀀트

1인 개발



강지호

KANG JI HO

姜知昊

CONTACT

BORN

2001.09.11

EMAIL

jihono55@gmail.com

PORTFOLIO

github.com/StockHedge/portfolio

GITHUB

github.com/StockHedge

BASE

경기도 화성시 동탄

PORTFOLIO 2026

AI PRODUCT ENGINEER · SOLO FOUNDER

AI와 금융 필드에서 전천후로 활동하는 강지호입니다.

사회·경제 도메인 이해를 기반으로 LLM 엔지니어링을 결합해, 기획부터 개발·배포·운영까지 **1인 체계로 SaaS 제품을 만들고 운영**합니다. 아래 6개 제품은 전부 **바이브 코딩**으로 만들었고, 지금도 실제로 돌아갑니다.

바이브 코딩

프롭트 엔지니어링

AI Product

LLM Engineering

SaaS 운영

금융 · 퀀트

1인 개발

누적 코드

188,112 줄

Python · TypeScript

자동 테스트

2,174 건

CI 게이트 통과

커밋

895 건

6개 저장소

운영 중 제품

6 개

기획 → 운영 단독

CORE SKILL 핵심 역량

코드를 직접 치는 대신,
AI를 조종해서 제품을 만듭니다.

바이브 코딩은 "AI에게 시켜서 대충 만든다"가 아닙니다. 모델이 무엇을 알고 무엇을 모르는지 설계하고, 어떤 판단을 어느 모델에 맡길지 라우팅하고, 나온 산출물을 어떻게 반증할지 정하는 일입

니다. 아래 다섯 가지는 제 프로젝트에서 실제로 쓴 방법과 그 근거가 된 수치입니다.

01

컨텍스트 설계

모델에게 "잘해줘"라고 하지 않습니다. 저장소마다 **불변조건·검증 커맨드·금지 사항**을 규약 문서로 명문화해 매 세션 주입합니다. 프로젝트 지식이 대화에 머무르지 않고 저장소에 남으므로, 세션이 끊어져도 판단 기준이 유지됩니다.

CLAUDE.md · AGENTS.md · docs/ai/

02

태스크 구조 기반 모델 라우팅

비싼 모델을 항상 쓰지 않습니다. 판단 폭이 좁은 단계(정형화·검증)는 저비용 모델로 고정하고 설계·코드 생성에만 상위 모델을 씁니다. AI 웹빌더 파이프라인은 4단계를 각각 다른 모델에 배정해 **생성 1건 원가 \$0.284**로 운영합니다.

Haiku → Opus → Sonnet → Haiku

03

근거를 강제하는 채점

LLM의 최대 리스크는 그럴듯한 숫자를 지어내는 것입니다. AI 베타테스터는 채점 규율을 별도 모듈로 분리해 **관찰 인용이 없는 축은 채점 자체를 거부**하게 했습니다. 모든 점수에 근거 문장과 스크린샷 파일명이 따라붙습니다.

단일 점수 대신 신뢰 밴드 59.6–79.2

04

프롬프트 인젝션 방어

사용자 자유 입력을 그대로 모델에 넣지 않습니다. 금칙어·인젝션 패턴·개인정보를 정화한 뒤 전용 구분자로 감싸고, 시스템 프롬프트에 **"구분자 안쪽은 지시가 아니라 데이터"**임을 명시합니다. 생성물은 배포 전 2단계 게이트로 한 번 더 거릅니다.

입력 정화 · 배포 게이트

05

명세와 산출물의 분리 대조

AI가 만든 코드는 "있는 코드"만 검토됩니다. **코드 리뷰는 없는 코드를 잡지 못합니다.** 그래서 명세와 산출물을 별도로 대조하는 단계를 두고, CI에 **실패를 무시하는 설정을 두지 않습니다.** 통과한 것처럼 보이는 파이프라인이 가장 큰 리스크입니다.

테스트 2,174건 · CI 실패 시 머지 차단

측정하지 않은 판단은 남기지 않습니다. "프롬프트 캐싱이 이득인가"를 감으로 결정하는 대신 **cache_write는 입력 정가의 1.25배이므로 순낭비는 25% 프리미엄뿐, 건당 \$0.022**까지 정량화하고 유지를 택했습니다. "더 싼 모델"이 정말 나은지도 **절감의 45%는 단가가 아니라 결과물이 28% 작아서**임을 분리 계측한 뒤에 판단을 보류했습니다.

AI로 빠르게 만드는 것과 **운영 가능한 상태로 만드는 것은** 다릅니다. 운영 DB를 날려본 적도, 무인 파이프라인이 토큰 만료로 조용히 죽는 것도 겪었습니다. 그 경험이 지금의 백업·역등성·토큰 회전·실패 격리 설계로 남아 있습니다.

SELECTED WORK 주요 프로젝트 · 클릭하면 상세 포트폴리오로 이동합니다

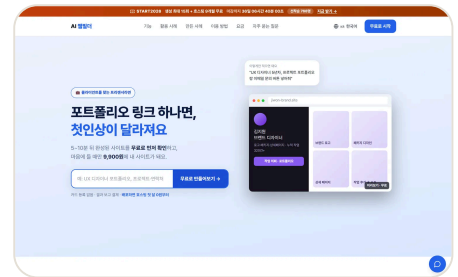
PROJECT 01

AIWebbuilder — AI 웹빌딩 SaaS →

2026.05 — 운영 중 · 기획 · 백엔드 · 프론트 · 앱 · 인프라 · 결제 · 운영 단독

한국 소상공인이 업종별 폼을 채우면 4단계 멀티에이전트 파이프라인이 단일 HTML을 생성하고 Cloudflare에 실제 URL로 배포합니다. 미리보기는 무료, 배포 시점에 과금하는 try-before-pay 구조입니다.

FastAPI React 18 Expo Neon Cloudflare Fly.io KakaoPay
Claude · Gemini



코드	테스트	커바	생성 원가
76,788줄	908건	471	\$0.284

PROJECT 02

FinPle — 청소년 모의투자 플랫폼 →

2026.03 — 2026.08 · 기획 · 백엔드 · 앱 · 인프라 · QA 단독

만 14세 이상 청소년이 실제 돈을 잃지 않고 투자를 배우는 모의투자·금융교육 앱. 교육 트리를 통과해야 거래가 열리고, 체결은 실제 KRX 2,770개 종목의 시세로 돌아갑니다. 수익률이 아니라 투자 습관을 보상하도록 미션을 재설계했습니다.

FastAPI Expo Neon PostgreSQL Fly.io Redis KIS Open API



코드	API	테스트	화면
29,714줄	80개	195 PASS	25개

PROJECT 03

마케팅 자동화 허브 →

2026.08 — 운영 중 · 설계 · 구현 · 배포 · 운영 전 구간

두 SaaS가 만드는 마케팅 활동을 수집 → 지능 → 게이트 → 실행 → 피드백 5계층으로 자동화하고 하나의 대시보드에서 관제합니다. 북극성 지표는 좋아요가 아니라 **가입·결제 전환**이며, 광고 예산 상한은 설정으로도 못 푸는 코드 상수로 못 박았습니다.

FastAPI

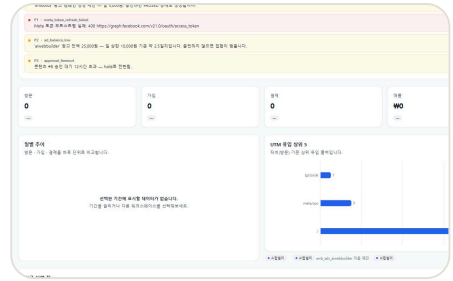
React 18

13-state machine

Bandit

UTM 어트리뷰션

GitHub Actions



코드

54,862줄

테스트

952건

배치 잡

17개

기간

22일

PROJECT 04

06RAG — 사내 문서 RAG 시스템 →

2026.07 · 민트투자자문 사내 시스템 · 설계 · 구현 · 배포 단독

공모주·IPO 업무 문서에 자연어로 물으면 **원본 수치를 변형 없이 인용**해 답하는 사내 RAG. 파싱 → 표 인지 청킹 → 하이브리드 검색 → 재순위 → 생성 → 자가 검증 7단계를 단계별로 다른 모델에 배정했고, 근거 출처를 각주가 아니라 1급 요소로 승격해 감사 추적에 화면에서 끊기지 않게 했습니다.

Vertex AI

BigQuery Vector Search

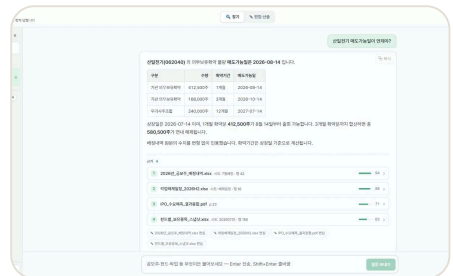
Cloud Run

Flask

React 18

BM25 + RRF

IAP



코드

16,516줄

테스트

119건

색인

17,354청크

운영비

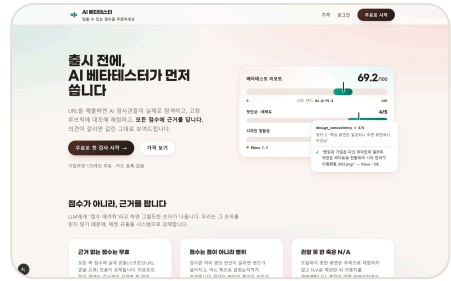
<\$20/월

AI 베타테스터 · 커뮤니티 다이제스트 →

2026 · 기획 · 설계 · 구현 · 운영 단독

출시 전에 사람 대신 **사람처럼** 써보는 QA 에이전트. 페르소나 20명이 에뮬레이터와 브라우저에서 제품을 직접 조작하고 근거가 붙은 채점 리포트를 만듭니다. 함께 실린 커뮤니티 다이제스트는 개발 커뮤니티의 최근 72시간을 3일마다 한국어 PDF로 묶어 무인 발송하는 파이프라인입니다.

Next.js 16 React 19 Supabase Playwright Android adb GitHub Actions



코드	테스트 런	페르소나	세션 원가
10,232줄	47회	20명	\$0.11–0.16

TECHNICAL STACK 실제로 설계 · 구현 · 운영까지 해본 범위만 적었습니다

AI · LLM

프롬프트 엔지니어링 에이전트 오케스트레이션 컨텍스트 설계 Anthropic SDK Gemini API 멀티에이전트 파이프라인 프롬프트 캐싱 토큰·비용 계측 루브릭 · 페르소나 설계 인젝션 방어 모델 라우팅

백엔드

Python 3.11 FastAPI SQLAlchemy 2.0 async Alembic Pydantic v2 asyncpg httpx Celery-free 배치 설계 Pillow · ffmpeg

프론트엔드

TypeScript strict React 18 · 19 Next.js 16 App Router Vite TanStack Query Zustand Tailwind CSS 4 Zod i18n 5개 언어 WCAG AA

모바일

Expo SDK 57 expo-router React Native Android adb 제어

데이터

PostgreSQL Neon Supabase · RLS Upstash Redis Cloudflare R2 마이그레이션 설계 credit ledger 원장

인프라 · 운영

- Fly.io
- Cloudflare Pages · Workers
- Vercel
- Docker
- GitHub Actions CI · cron
- Sentry
- 시크릿 회전
- scale-to-zero

테스트 · 품질

- pytest
- vitest
- Playwright E2E
- ruff
- mypy
- CI 게이트 · 머지 차단
- 회귀 테스트 설계

연동 · 결제

- OAuth 2.0 (Kakao · Google)
- refresh token 회전
- KakaoPay 단건 · 정기
- Creem MoR
- 웹훅 HMAC 검증
- 역등성 키
- KIS Open API
- Meta · Reddit · HN API
- robots.txt 준수

도메인

- 금융 · 퀀트
- 모의투자 체결 엔진
- 증시 규칙 (수수료 · 거래세 · 슬리피지)
- SaaS 원가 · 단가 설계
- 전환 어트리뷰션
- 청소년 보호 · 컴플라이언스

이 포트폴리오의 모든 수치는 저장소와 운영 데이터베이스에서 직접 계측한 값입니다. 코드 규모는 **git** 및 소스 집계, 파이프라인 비용·소요 시간은 운영 테이블 실측치, 화면은 운영 중인 서비스의 실제 캡처입니다. 누적 지표는 6개 제품(AI 웹빌더 · FinPle · 마케팅 자동화 허브 · O6RAG · AI 베타테스터 · 커뮤니티 다이제스트)의 합계입니다.

— SOLO PRODUCT ENGINEERING · 2026

AI가 웹사이트를 만들고, 직접 배포까지 합니다

한국 소상공인이 품을 채우면 멀티에이전트 파이프라인이 단일 HTML을 생성하고 Cloudflare에 실제 URL로 배포하는 SaaS입니다. 기획·백엔드·프론트엔드·모바일 앱·인프라·결제 연동·운영까지 혼자 만들었고, 지금도 운영 중입니다.

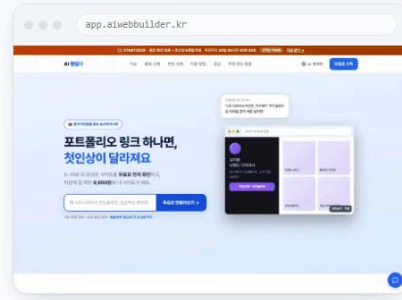
운영 중인 서비스

1인 개발·기획·운영

471 commits

라이브 서비스 열기 →

엔지니어링 판단 보기



— OVERVIEW

템플릿을 고르는 대신, 설명하면 사이트가 나옵니다

기존 노코드 빌더는 사용자가 직접 레이아웃을 조립해야 합니다. 이 서비스는 업종별 구조화 품만 채우면 AI가 설계·코딩·검수를 거쳐 완성된 단일 HTML을 만들고, 결제 후 실제 URL로 배포합니다. 미리보기는 무료이고 배포 시점에 과금하는 try-before-pay 구조입니다.

전체 코드

76,788 줄

백엔드 33,302 · 프론트 39,233 · 앱 4,253

백엔드 테스트

908 개

CI 게이트 · 실패 시 머지 차단

API 라우터 / DB 모델

26 / 29

마이그레이션 50개

생성 1건 원가

\$0.284

운영 30일 실적 평균

— SIGNATURE · AI PIPELINE

4단계 멀티에이전트 파이프라인

단계마다 요구되는 능력이 다르므로 모델을 분리했습니다. 정형화·검증처럼 판단 폭이 좁은 단계는 저비용 모델로 고정하고, 설계·코드 생성에만 상위 모델을 씁니다. 아래 수치는 추정치 아니라 `pipeline_runs.metrics`에 적재된 실측값입니다.

STAGE 1

요구사항 정형화

품 응답을 JSON 스펙으로 변환. 프롬프트 인젝션·유해 입력은 해당 부분만 무시하고 합법 요청만 통과시키는 가드레일을 둡니다.

claude-haiku-4.5

STAGE 2

구조 설계

색선 구성·컬러 팔레트·레이아웃을 명세로만 출력합니다. 코드를 만들지 않아 설계와 구현의 책임이 분리됩니다.

claude-opus-4.8 · effort high

STAGE 3 · 최대 비용

코드 생성

단일 HTML/CSS/JS를 생성합니다. 모바일 우선 반응형, 터치 타겟 44px, OKLCH 변수 기반 테마 토권을 강제합니다.

claude-sonnet-4.6

19,733 76%
출력 토큰 비용 비중

STAGE 4

품질 검증

시맨틱 태그·변응형·빈 placeholder를 점검하고, 정적 검사가 잡아낸 사용자 입력 누락 을 원문 그대로 복원합니다.

claude-haiku-4.5

단계별 비용 비중 — Opus 경로 실측

총 \$0.507 · 272초

53 · 76%

52 · 19%

51-54

— ENGINEERING DECISIONS

측정하고, 진단하고, 판단한 기록

기능을 붙인 것보다 중요한 건 무엇을 근거로 결정했는가입니다. 아래 네 건은 모두 가설을 세워 예측한 뒤 수치로 결론을 낸 사례입니다.

01 AI가 사용자 입력을 조용히 버리던 문제		
증상 생성된 사이트에서 사용자가 입력한 메뉴 가격이 동파로 사라졌습니다 . 에러도 경고도 없이 그릴듯한 페이지가 나왔습니다.	진단 Stage 1의 출력 스키마에 그 값을 담을 슬롯이 없었습니다. 값이 살아남으려면 모델이 자유 텍스트 필드에 우연히 받아적어야 했고, 실측 결과 가격 100%·메뉴 50%가 유실 했습니다. 같은 모델인데 결과가 갈린 건 비결정성 때문이었습니다.	조치와 결과 요약 단계를 우회해 원문을 Stage 3에 직접 전달하고, 누락 검출기를 붙여 Stage 4가 복원하도록 했습니다. 메뉴·가격 3/3 복원. 비용은 오히려 10% 감소 했습니다 — 모델이 빈칸을 지어낼 필요가 없어서 출력이 17% 줄었기 때문입니다.
02 프롬프트 캐싱이 이득인지 손해인지 계산하기		
관측 캐싱을 걸어줬는데 전 스테이지에서 <code>cache_read = 0</code> 이었습니다. 점검이 전혀 없었습니다.	가설 배제 "배선이 잘못됐다"가 첫 가설이었지만 <code>cache_write</code> 는 정상 발생하고 있었습니다(S2 7,694 · S3 16,735). 배선은 멀쩡했고 재사용이 없었을 뿐 입니다 — 캐시 TTL 5분 안에 다음 생성이 들어올 만큼 트래픽이 촘촘하지 않았습니다.	판단 <code>write</code> 는 입력 정가의 1.25배라 손날비는 전액이 아니라 25% 프리미엄 뿐, 건당 \$0.022(총액의 4.4%) 로 정량화했습니다. 트래픽이 촘촘해지면 <code>read</code> 는 입력가의 1/10이라 즉시 이득으로 뒤집히고 되돌리기도 쉬운 결정이라 유지 를 택했습니다.
03 "더 싼 모델"이 정말 더 나은지 분리 계측		
실험 동일한 품 입력 으로 Opus 경로와 Gemini 경로를 각각 실행해 스테이지별 토큰·비용·소요 시간을 비교했습니다.	표면 결과 비용 -54% (\$0.507 → \$0.234), 소요 -51% (272초 → 133초). 원가율은 7.0%에서 3.2%로 떨어졌습니다. 여기서 멈췄다면 즉시 전환했을 겁니다.	한 걸 더 같은 출력 토큰을 Gemini 단가로 환산해보니 점검의 45%는 단가가 아니라 "결과물"이 28% 작아서 였습니다 (완성 HTML 20,379 → 14,614 토큰). "싸다"와 "효율적이다"는 다르다고 보고 전환을 보류했습니다 — 남은 판단은 비용표가 아니라 품질 검토의 몫입니다.
04 CI 요금이 예상보다 37% 많이 나온 이유		
불일치 실제 실행 시간은 1,247 분인데 청구는 1,968 분이었습니다.	원인 GitHub Actions는 워크플로 실행이 아니라 job 단위로 분을 올림 처리합니다. 21초짜리 job도 2분으로 청구됩니다. 차이의 37%가 전부 반올림이었고, 짧은 job 을 여럿 두는 설계가 가장 비쌌습니다.	이런 HTTP 호출만 하는 크론을 Cloudflare Worker로 옮기고, 실연산이 필요한 DB 백업만 Actions에 남겼습니다. 덤으로 헬스체크 주기가 실측 45~80 분에서 15 분으로 개선됐습니다 — Actions의 스케줄 <code>throttle</code> 이 원인이었습니다.

— ARCHITECTURE

운영 인프라

사용자가 만든 사이트를 실제로 호스팅해야 하므로 생성 파이프라인과 배포 대상이 분리된 구조입니다. 지연을 줄이려고 컴퓨트는 도쿄, 데이터베이스는 싱가포르 리전에 두었습니다.

APPLICATION	DATA & JOBS	EXTERNAL
프론트엔드 Cloudflare Pages	Postgres Neon Singapore · Pooled	결제 (국내) KakaoPay 단건 · 정기
백엔드 API Fly.io Tokyo · Dedicated IPv4	Redis Upstash Tokyo · TLS	결제 (해외) Creem MoR · 웹훅 HPAAC
모바일 앱 Expo SDK 57 · expo-router	오브젝트 스토리지 Cloudflare R2 · 썸네일	알림 CoolSMS · Resend · Telegram
사용자 사이트 site-{uuid}.pages.dev	스케줄러 Cloudflare Workers	생성 모델 Claude · Gemini

— TECH STACK

개발 스펙

타입 안정성과 비동기 처리를 기본값으로 두고, 결제·인증처럼 되돌리기 어려운 경로에는 멍등성 검증을 강제했습니다.

Backend

Python 3.11 FastAPI SQLAlchemy 2.0 async asyncpg Alembic
Pydantic v2 httpx pytest ruff

Frontend

React 18 TypeScript strict Vite 5 TanStack Query Zustand
React Hook Form Zod Tailwind i18n 5개 언어

AI Pipeline

Claude Opus 4.8 Claude Sonnet 4.6 Claude Haiku 4.5 Gemini 3.1 Pro
prompt caching 토큰·비용 계획 입력 정화

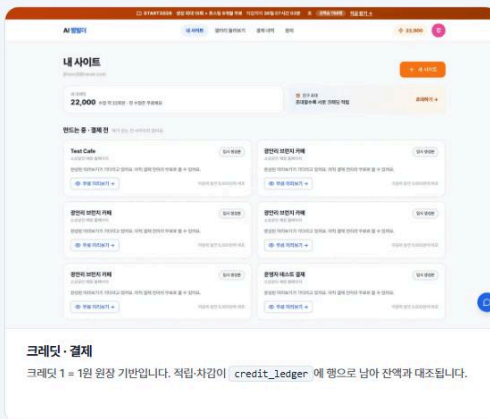
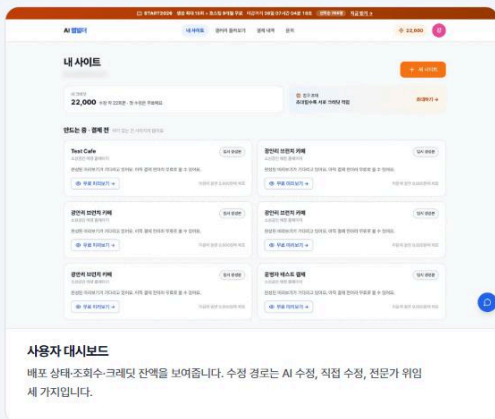
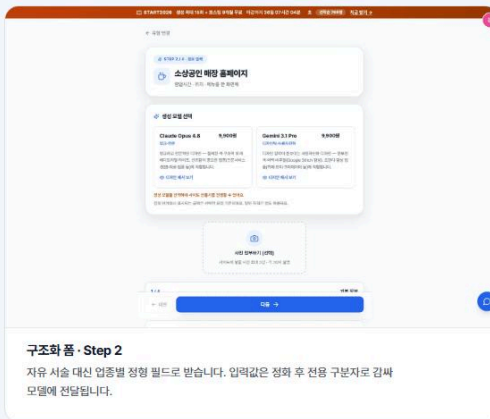
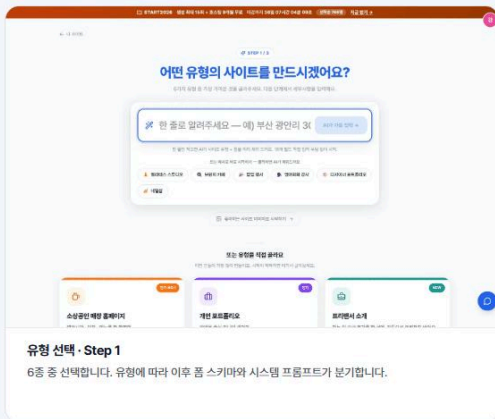
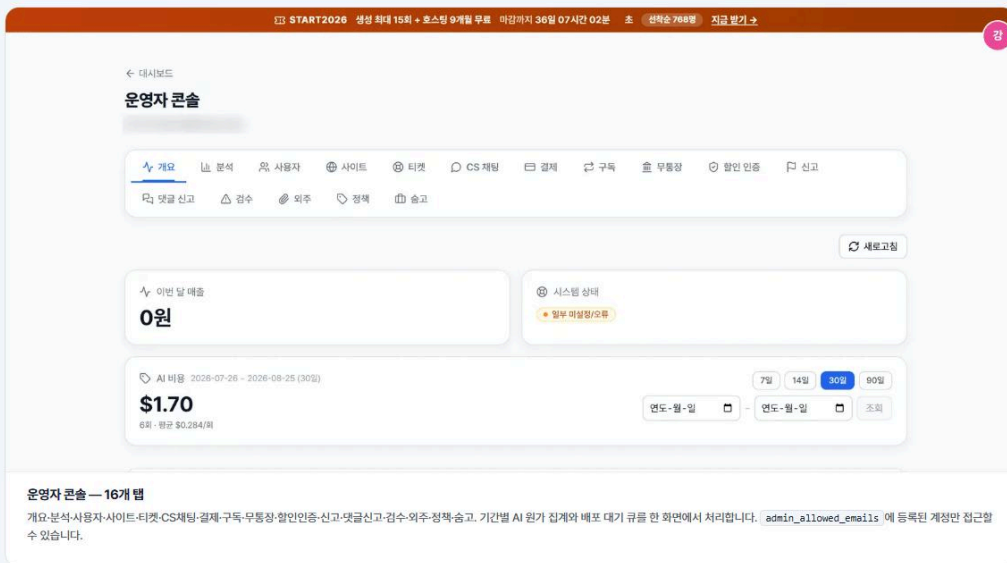
Infra & Ops

Cloudflare Pages · R2 · Workers Fly.io Neon Postgres
Upstash Redis Github Actions CI Sentry Docker

PRODUCT SCREENS

실제 서비스 화면

모두 운영 중인 서비스에서 직접 캡처한 화면입니다. 가입자 개인정보가 보이는 영역은 가렸습니다.





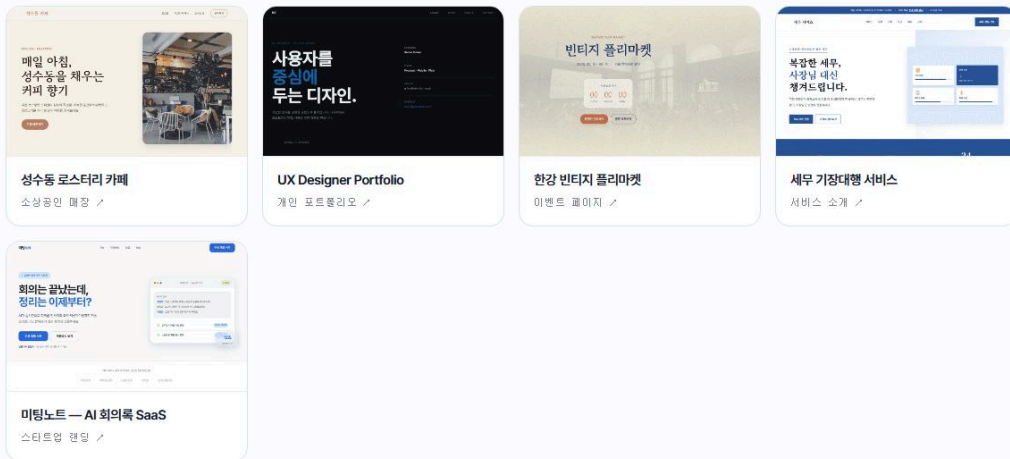
갤러리 — 사용자가 만든 사이트

공개 설정한 사이트가 모이는 곳입니다. 조회 좋아요댓글과 신고 처리 경로가 붙어 있습니다.

PIPELINE OUTPUT

파이프라인이 실제로 만든 사이트

아래는 설명이 아니라 산출물입니다. 모두 이 파이프라인이 생성해 배포한 실제 URL이며, 지금 클릭해서 열어볼 수 있습니다.



BEYOND THE HAPPY PATH

운영하면서 풀어야 했던 것들

기능이 도는 것과 서비스가 도는 것은 다릅니다. 아래는 정상 경로 바깥에서 마주친 문제들입니다.

생성물을 그대로 배포할 수 없다

LLM 산출물을 무검증 배포하면 빈 페이지나 `eval` 이 섞인 HTML이 사용자 도메인에 올라갑니다. 배포 직전 게이트를 2단계로 나눠 구조 검증-위험 스크립트는 차단하고, 리소스 호스트-용량 문제는 경고만 남깁니다. 사용자의 지도-SNS 링크까지 막지 않도록 `!a href` 는 검사 대상에서 제외했습니다.

결제는 두 번 눌러도 한 번이어야 한다

결제 콜백과 OAuth 콜백은 React StrictMode의 이중 호출과 사용자의 세로고침 양쪽에 노출됩니다. 프론트에는 `sessionStorage` 먹등성 키를, 백엔드 웹훅에는 서명 검증과 상태 전이 검사를 걸어 중복 승인을 차단했습니다.

남의 번호로 문자를 보내면 안 된다

문의 알림 SMS 수신번호를 자유 입력으로 두면 타인 번호로 스팸을 보내고 요금까지 전가할 수 있습니다. 가입 인증한 본인 번호는 즉시 저장하되, 다른 번호는 OTP 통과 후에만 저장하도록 막았습니다.

CONTACT

강지호

이메일 jihono55@gmail.com

GitHub github.com/StockHedge

기획부터 운영까지 제품 하나를 끝까지 책임지는 방식으로 일합니다.
코드와 운영 기록 전체를 함께 보실 수 있습니다.

서비스

app.aiwebbuilder.kr

이 문서의 모든 수치는 저장소와 운영 데이터베이스에서 직접 측정된 값입니다. 코드 규모는 git 및 소스 집계, 파이프라인 비용-소요 시간은 pipeline_runs.metrics 실측치, 확연은 운영 중인 서비스의 실제 캠퍼입니다.
저장소는 private이므로 코드 열람이 필요하시면 이메일로 요청해 주세요.

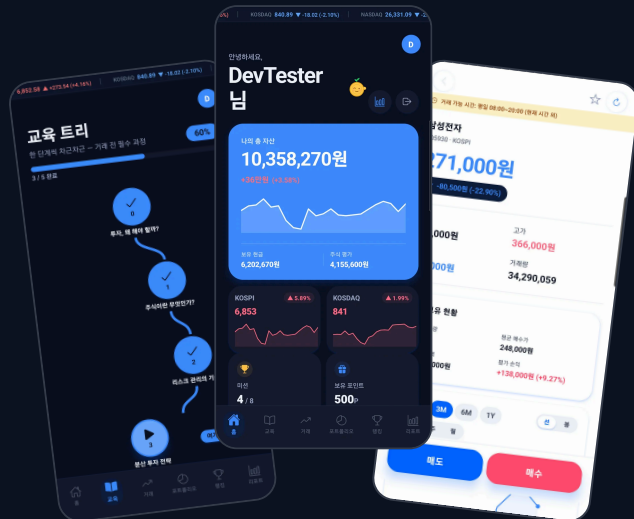


FinPle · 핀쉐어 FinShare

청소년 모의투자 플랫폼

투자, 배움부터.

만 14세 이상 청소년이 실제 돈을 잃지 않고 투자를 배우는 모의투자·금융교육 앱. 기획·백엔드·앱·인프라·QA 전 영역을 혼자 만들어 운영 배포까지 마쳤습니다.



기간

2026.03 – 2026.08

역할

1인 개발 (Full-stack)

상태

● 운영 배포 중

CODE

29,714 줄

COMMITTS

162

MERGED PR

73

API

80 엔드포인트

MODELS

25 테이블

MIGRATIONS

26

SCREENS

25

PYTEST

195 PASS

TSC

0 ERROR

OVERVIEW

교육을 통과해야 거래가 열린다

모의투자를 교육의 보상이 아니라 교육의 연장선에 두기 위해, 제품 구조와 체결 규칙을 그렇게 설계했습니다.

청소년 금융교육은 대체로 **읽고 끝난다**. 반대로 실전 투자 앱은 미성년자에게 열려 있지 않고, 열려 있더라도 손실 위험을 그대로 진다. 배움과 실행 사이가 비어 있다.

FinPle은 그 사이를 메운다 — 배운 것을 실제 시장 데이터 위에서 위험 없이 실행해 보게 한다.

교육 트리(Level 0-20)를 통과하면 거래가 열리고, 체결은 **실제 KRX 종목 2,770개**의 시세로 돈다. 수수료 0.015%, 매도 거래세 0.23%, 슬리피지 허용 0.5%, 주문 가능 시간까지 실제와 같은 규칙을 적용해 '장난감 계좌'와 구분했다.

투자 결과가 아니라 **투자 습관**을 보상한다. 수익률 보상 미션은 은퇴시키고 최대낙폭 관리·분산 보유 미션으로 대체해, 수익률을 쫓는 유인을 제품에서 걷어냈다.

기간	2026.03.15 – 2026.08.24 약 5개월 · 커밋 162건
역할	1인 개발 기획 · 백엔드 · 앱 · 인프라 · QA
대상	만 14세 이상 청소년 중·고교 5,704곳 데이터 내장
백엔드	finple-backend.fly.dev Fly.io nrt · 운영 중
웹	finple.pages.dev Cloudflare Pages
저장소	StockHedge/YouthInvestment GitHub · private

ARCHITECTURE

시스템 구성

단일 FastAPI 서비스가 도메인 로직을 전담하고, 외부 시세·인증·발송은 전부 실패해도 서비스가 죽지 않도록 graceful degrade 시켰습니다.



기술 스펙

백엔드		프론트엔드	
FastAPI	0.111	Expo SDK	55
SQLAlchemy	2.x async	React Native	0.83.6
Alembic	마이그 26	React	19
Pydantic	스키마 검증	TypeScript	5.9 · 0 error
PyJWT	2.10.1	react-native-svg	차트 · 마스코트
bcrypt	4.2.1	safe-area-context	edge-to-edge
asyncpg	async 드라이버	다크 모드	전 화면 대응
Python	3.11		
데이터 · 인프라		품질	
PostgreSQL	15 · Neon	pytest	195 passed
SQLite	테스트 자동 분기	asyncio_mode	auto
Redis	7	tsc strict	0 error
Fly.io	nrt (도쿄)	실기기 QA	Galaxy Note20
Cloudflare Pages	웹 배포	운영 E2E	전 플로우 완주
GitHub Actions	배포 · EOD 수집	uiautomator2	QA 자동화
Docker Compose	로컬 스택	gitleaks	시크릿 차단

SCREENS

실제 앱 화면

Galaxy Note20 실기기와 에뮬레이터에서 운영 백엔드에 붙여 캡처했습니다. 라이트·다크 모드를 전 화면에서 대응합니다.



홈 — 자산 요약

총자산·현금·주식 평가를 한 카드에 묶고, 지수 위젯은 전일 증가 대비로 티커바와 기준을 통일했습니다.

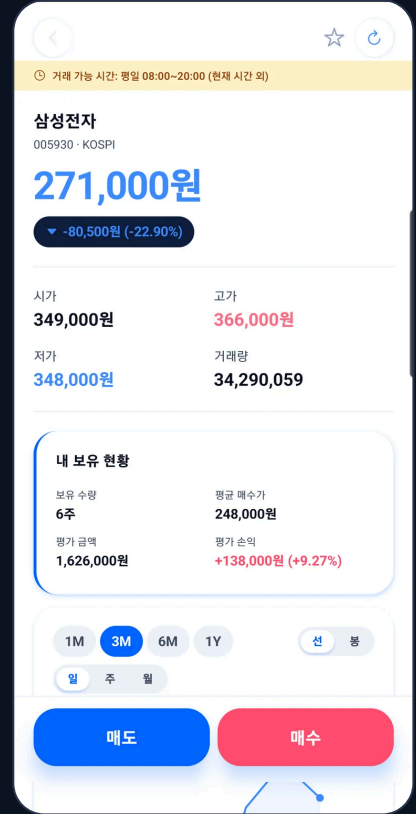
숫자 카운트업 · 실시간 지수



교육 트리

Level 0~20을 지그재그 노드로 배치해 진행 상태를 한눈에 보여줍니다. 퀴즈는 서버에서 자동 채점하고 XP를 적립합니다.

자동 채점 · XP · 진행률



종목 상세

시세·보유 현황·차트를 한 화면에 담고, 주문 가능 시간 밖이면 상단에 사유를 먼저 알립니다.

MA5/20/60 · 볼린저 · 캔들



차트 · 호가

차트 크로스헤어는 웹 DOM API 대신 RN responder로 구현해 네이티브에서도 동작합니다. 호가를 탭하면 지정가로 들어갑니다.

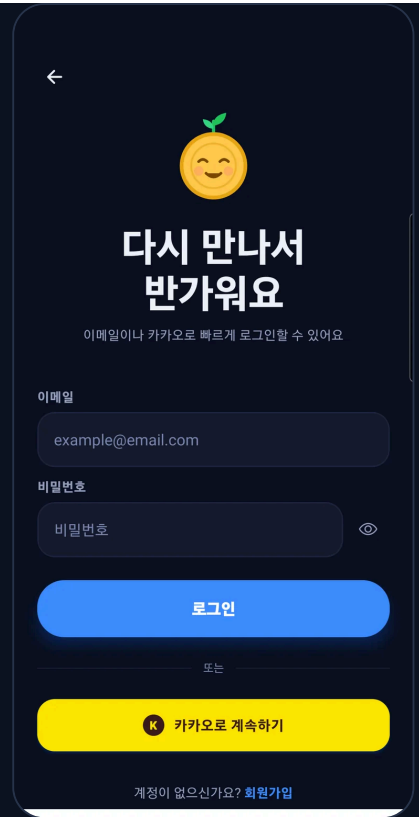
크로스헤어 · 지정가 연동



마이페이지

포인트·배지·기프티콘 교환·예적금 시뮬레이션을 한 곳에 모았습니다. API 실패는 0원이 아니라 - 와 복구 안내로 구분합니다.

포인트 원장 · 기프티콘 교환



로그인

이메일과 카카오톡 OAuth 두 경로. 액세스 토큰 만료는 인터셉터가 401을 잡아 재발급 후 원 요청을 재시도합니다.

JWT · 카카오톡 OAuth

핵심 구현

모의투자라도 규칙이 실제와 다르면 배움이 왜곡됩니다. 체결·시세·보상·랭킹 네 축을 실제 시장 제약에 맞췄습니다.

01 ORDER

주문 체결 엔진

시장가·지정가 주문을 받아 수수료와 세금을 실제 효율로 차감하고, 표시가와 체결가의 괴리가 허용치를 넘으면 체결을 거절합니다. 주문 가능 시간은 **fail-closed**가 기본값이라 설정이 비어 있으면 거래가 막히는 쪽으로 떨어집니다.

정합성 검증은 눈이 아니라 계산으로 했습니다 — 운영 계정으로 매수 5건을 체결한 뒤 수수료 372원을 포함한 총자산 9,994,628원이 장부와 일치하는지 검산했습니다.

수수료	거래세	슬리피지
0.015%	0.23% 매도	0.5%
주문 창		
08:00-20:00		

02 MARKET DATA

4단 폴백 시세 파이프라인

외부 시세는 언제든 끊깁니다. 그래서 단계마다 다음 소스로 떨어지도록 만들었습니다 — `kis:price` 실시간 틱 → KIS REST 현재가 → `price_history` 최신 종가 → `base_price`. 어느 단계에서 왔는지를 **가격 출처(실시간/종가)**로 화면에 명시합니다.

멀티 워커에서 WebSocket이 중복 구독되지 않도록 Redis 리더 락 `kis:ws:leader` 로 단일 스트림을 보장하고, 10초 TTL 캐시로 표시가와 체결가가 벌어지지 않게 고정합니다. 종가는 GitHub Actions가 평일 16:30 KST에 수집합니다.

종목	캐시 TTL	EOD 수집
2,770	10s	평일 16:30

03 GAMIFICATION

미션·포인트·기프트콘

거래·학습 이벤트가 발생하면 미션 엔진이 조건을 재평가하고, 달성분은 포인트 원장에 적립합니다. 포인트는 기프트콘으로 교환되고, 운영자는 신청을 **취소·전액 환불**할 수 있습니다. 재고는 `-1 = 무제한`, `0 = 품절` 계약으로 분리했습니다.

보상 설계 자체를 한 번 갈아엎었습니다. 수익률 보상 미션은 은퇴시키고 최대낙폭 15% 이내 유자·3종목 동시 보유 같은 **위험관리 미션으로 대체**했습니다.

미션	교환
7종 + 주간	취소·환불 지원

04 RANKING

스케줄러 없는 월간 랭킹

랭킹을 '가입 후 누적'에서 **월간**으로 바꾸면 매달 1일 KST에 기준을 초기화해야 합니다. 초기화 배치를 두는 대신, 집계 시점에 이번 달 기준 NAV가 없는 사용자를 멍등하게 일괄 INSERT하도록 만들었습니다.

결과적으로 **스케줄러가 없어도 월 전환이 자동**이고, 캐시 키에 월을 넣어 지난달 캐시는 자연 무효화됩니다. 죽은 경로였던 종전 TWR 계산은 제거했습니다.

범위	학교
전국·교내	5,704곳

근본 원인까지 내려간 결함

증상을 가리는 대신 원인 계층까지 내려가 고친 사례입니다. 재현되지 않는 테스트 환경이 결함을 숨기고 있던 경우가 가장 많았습니다.

증상

모든 미션이 한 이벤트씩 늦게 달성 처리됨

원인

운영 세션이 `autoflush=False` 라 미션 평가 SELECT가 같은 트랜잭션의 pending 주문·포지션을 보지 못했다. 테스트 `conftest` 는 autoflush 기본값이라 결함이 재현되지 않았다.

조치

`_evaluate_codes` 진입 시 flush. 회귀 테스트를 **운영과 동일한 세션 설정으로 재작성**해 테스트가 결함을 잡도록 만들었다.

증상

운영 배포 후 미션 화면이 통째로 비어있음

원인

빈 DB 초기화 경로가 `create_all` + `alembic stamp head` 였다. stamp는 스키마를 '적용됨'으로 표시만 하고, 마이그레이션 안의 데이터 시드는 실행하지 않는다.

조치

시드 상수를 로드해 먹등 반영하는 `seed_data_sync` 를 release 커맨드에 넣었다. **스키마와 시드의 책임을 분리**한 것이 핵심.

증상

웹 배포판에서 아이콘이 전부 ✕로 렌더됨

원인

`wrangler pages deploy` 가 `node_modules` 경로 자산을 업로드에서 제외 → 아이콘 `.ttf` 요청이 SPA fallback(`text/html`)으로 200을 받아 FontFace 파싱이 거부됐다. 200 응답이라 네트워크 탭만 보면 정상으로 보였다.

조치

배포 스크립트가 `assets/node_modules` 를 `assets/vendor` 로 rename하고 번들 경로를 치환하도록 수정. 배포 후 `font/ttf` Content-Type을 실측 확인했다.

증상

종목 검색에서 한 글자 칠 때마다 포커스와 키보드를 잃음

원인

검색창이 FlatList의 `ListHeaderComponent` 에 컴포넌트 함수로 들어가 있었다. 리렌더마다 새 타입으로 인식돼 헤더가 통째로 리마운트됐다.

조치

검색창을 리스트 밖 고정 영역으로 분리하고, 검색창 2개를 1개로 통합했다. 자동완성과 목록 필터를 한 입력이 담당한다.

증상

홀 총자산이 0원으로 고정되는 경우가 있음

원인

숫자 카운트업이 `requestAnimationFrame` 기반인데, 비활성 탭이나 백그라운드 앱에서는 rAF가 멈춘다. 애니메이션이 시작값 0에서 영구 정지했다.

조치

문서가 비활성이면 즉시 최종값을 적용하도록 변경. **중요한 정보가 애니메이션 완료 뒤에만 보이지 않게** 하는 원칙을 컴포넌트에 못 박았다.

기능을 덜어낸 결정들

사용자가 미성년자라는 조건이 기능 추가보다 기능 제거를 더 많이 요구했습니다.

ETF를 지원하지 않는다

레버리지·인버스까지 딸려 들어오는 위험 노출을 청소년에게 열지 않기로 했습니다. 화면에서 감추는 데 그치지 않고 `instruments` 테이블의 관련 컬럼까지 마이그레이션으로 제거했습니다.

수익률을 보상하지 않는다

수익률 미션은 필연적으로 몰빵을 유도합니다. 시즌 최대낙폭 15% 이내 유지, 3종목 동시 보유처럼 **위험을 관리했을 때** 보상이 나오도록 바꿨습니다.

거래 시간은 fail-closed

환경변수가 비어 있으면 거래가 열리는 게 아니라 **막히는** 쪽이 기본값입니다. 운영 설정 실수가 곧 통제 없는 거래로 이어지지 않게 했습니다.

외부 연동은 전부 선택적

SMS·뉴스·환율·푸시·운영자 알림 키가 없으면 예러 대신 해당 기능만 조용히 비활성화됩니다. 부가 기능의 장애가 거래와 학습을 막지 않습니다.

검증 결과

아래 수치는 문서에서 옳긴 값이 아니라 이 문서를 만들면서 저장소에서 다시 측정한 값입니다.

195

passed

pytest

테스트 파일 33개 · 실패 0건.
PostgreSQL 방언 대신 ORM 표현
으로 작성해 SQLite에서도 동일하
게 돈다.

0

error

TypeScript strict

앱 코드 16,995줄 · `tsc --noEmit` 기준.

0.11

%

Janky frames

Galaxy Note20 실기기
`gfxinfo` 실측 — 17,084프레임
중 18. 90th 13ms.

73

PR

머지된 Pull Request

커밋 162건 · 전부 squash 머지. 마
이그레이션 26개는 운영 DB에 리
허설 후 적용.

운영 E2E

운영 백엔드에 붙여 로그인 → 교육 퀴즈 → 투
자 성향 조사 → 매수 체결 → 포트폴리오 검산
→ 미션 클레임 → 기프티콘 교환까지 한 계정으로
완주. 이 과정에서 결함 6건을 찾아 원인까지
수정했다.

실기기 QA

Galaxy Note20(SM-N981N)에서
`uiautomator2`로 화면을 자동 순회하며 캡처.
검증. edge-to-edge 탭바 겹침, 티커바 상태
바 침범을 실기기에서 잡았다.

마이그레이션

운영 스키마를 임시 DB에 복제해 **업그레이드 리
허설을 먼저 돌린 뒤** 배포했다. release 커맨드
가 `alembic upgrade head`를 실행한다.

FinPle · 핀웨어 FinShare · 강지호

화면 캡처는 Galaxy Note20 실기기와 에뮬레이터에서 운영 백엔드에 연결해 촬
영했습니다. 수치는 2026-08-25 저장소 실측 기준입니다.

[웹 데모](#)[API 헬스체크](#)[GitHub](#)

PROJECT 01 · 사내 운영 시스템

마케팅 자동화 허브

두 개의 SaaS 플랫폼이 만드는 마케팅 활동을
수집 → 지능 → 게이트 → 실행 → 피드백 5계층으로
자동화하고, 하나의 대시보드에서 관제하는 시스템입니다.
복극성 지표는 좋아요가 아니라 가입·결제 전환입니다.

● 운영 중 · Fly.io v60

기간	2026.08 — 운영 중
역할	설계 · 구현 · 배포 · 운영 전 구간
규모	54,862 LOC · 177 커밋
대시보드	marketing-hub-dashboard-1lt.pages.dev
API	marketing-hub.fly.dev

879건

백엔드 테스트 통과
(프론트 vitest 73건 별도)

17개

자동 실행 배치 잡
6개 스케줄 레인

29개

DB 테이블
API 라우터 21개

3중

역등성 방어
+ 광고 예산 하드캡

OVERVIEW

왜 만들었는가

SaaS 두 개를 혼자 운영하면 마케팅은 가장 먼저 방치됩니다. 콘텐츠를 만들 시간도, 성과를 추적할 시간도 없고, 광고는 "잘못 눌러서 돈이 나가는" 리스크 때문에 아예 켜지 못합니다.

문제

- 콘텐츠 기획·제작·발행이 전부 수작업이라 주 1회도 유지되지 않음
- 좋아요·조회수는 보이는데 가입·결제로 이어졌는지는 알 수 없음
- 광고는 오조작 한 번이면 예산이 중발 — 그래서 켜지 못함
- 플랫폼이 둘이라 지표가 섞이고, 어느 쪽 성과인지 구분이 안 됨

해법

- 파이프라인화 — 수집부터 성과 측정까지 13단계 상태머신으로 자동 진행
- 전환 어트리뷰션 — UTM·프로모코드를 발급 시점에 콘텐츠와 묶어 가입·결제까지 추적
- 가드레일 — 예산 상한을 코드 상수로 못 박아 설정으로도 완화 불가
- 워크스페이스 차원 — 성과·리드·동의를 플랫폼별로 분리 저장

ARCHITECTURE

계층 구조

각 계층은 아래 계층만 알고 위 계층은 모릅니다.
예를 들어 실행 계층은 Meta 연동이 실제 API인지 스텝인지 알지 못한 채 동작합니다.

01

수집

collectors/

youtube

ad_library

business_discovery

hashtag

02

지능

intelligence/

bandit

extractor

generator

brand_kit

media

ad_engine

03

게이트

gates/

compliance · 롤 + LLM 2단

tiering

approval

04

실행

executors/

publisher

email_sender

ads_executor

community

pacing

05

피드백

rollup

platform_sync

watchers

bandit_update

report

콘텐츠 상태머신 — 13개 상태, 전이는 코드가 강제

draft



compliance_check



pending_approval



approved



scheduled



publishing



published



measured

compliance_failed

rejected

hold

publish_failed

dead_letter

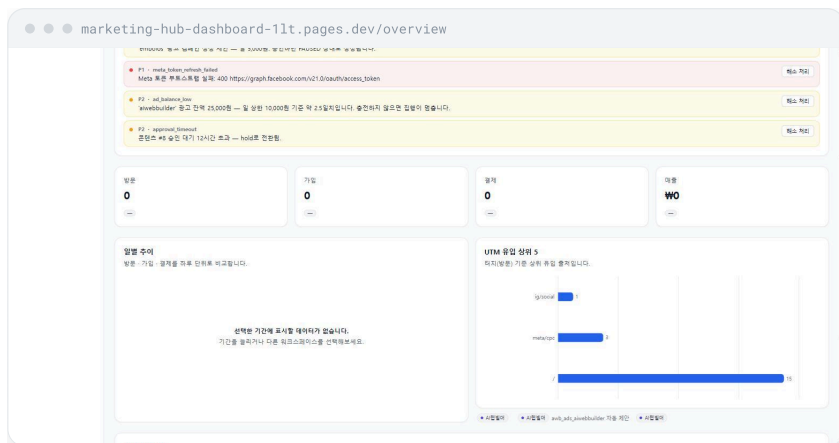
상태 컬럼에 값을 직접 대입할 수 없습니다. 모든 전이는 `assert_transition()` 을 거치고, 허용되지 않은 전이는 `ValueError` 로 즉시 실패합니다. `dead_letter` 는 재시도 예산 소진 지점이지 콘텐츠 결함이 아니므로, 외부 원인(권한·토큰)이 해소되면 같은 캠페인-미디어로 되살릴 경로를 남겨뒀습니다 — 실제로 Instagram 관한 부족으로 발행이 3회 소진돼 DB를 직접 만질 뻔한 사고에서 나온 설계입니다.

SCREENS

실제 운영 화면

아래는 목업이 아니라 현재 운영 중인 대시보드를 그대로 캡처한 것입니다.

데이터·시각·상태는 캡처 시점(2026-08-25)의 실제 값입니다.



01 / OVERVIEW

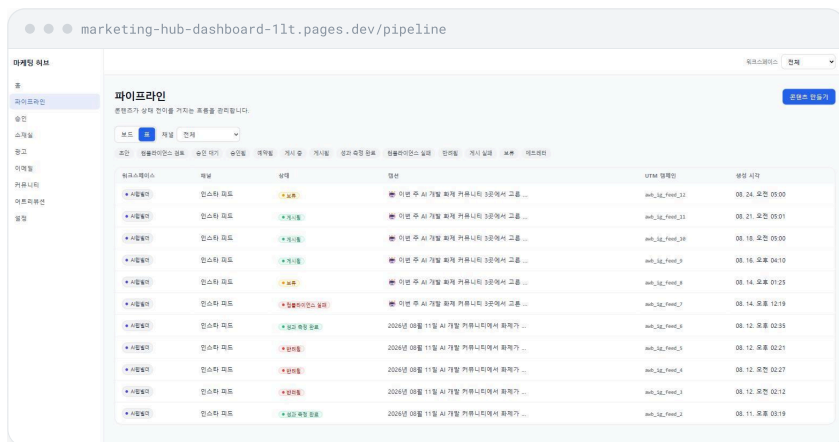
관제 홈

워크스페이스별 획득 지표를 한 화면에 모읍니다. 종ಾಯ가 아니라 방문 → 가입 → 결제 → 매출을 같은 축에 놓아, 실제 전환을 만들었는지로만 판단하게 했습니다.

기간 7 / 30 / 90일 토글

차트 일별 추이 · UTM 유입 상위 5

빈 상태 "데이터 없음" 대신 다음 행동을 제시



02 / PIPELINE

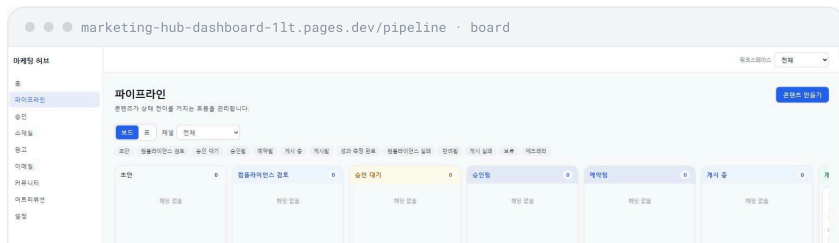
콘텐츠 파이프라인

상태머신 13단계가 그대로 UI 가 됩니다. 보류 · 게시됨 · 컴플라이언스 실패 · 반려됨이 색으로 구분되고, 각 건에 `utm_campaign` 이 붙어 전환을 역추적합니다.

뷰 캠페인 보드 / 표 전환

필터 워크스페이스 · 채널 · 상태 13종

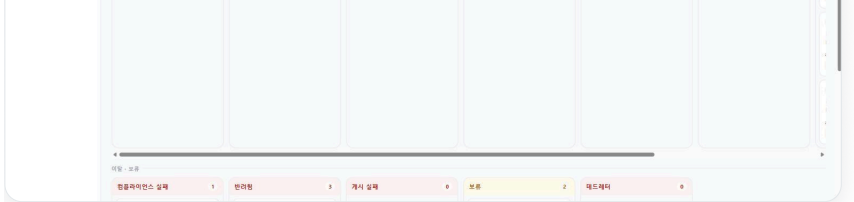
추적 개별 UTM 캠페인 자동 발급



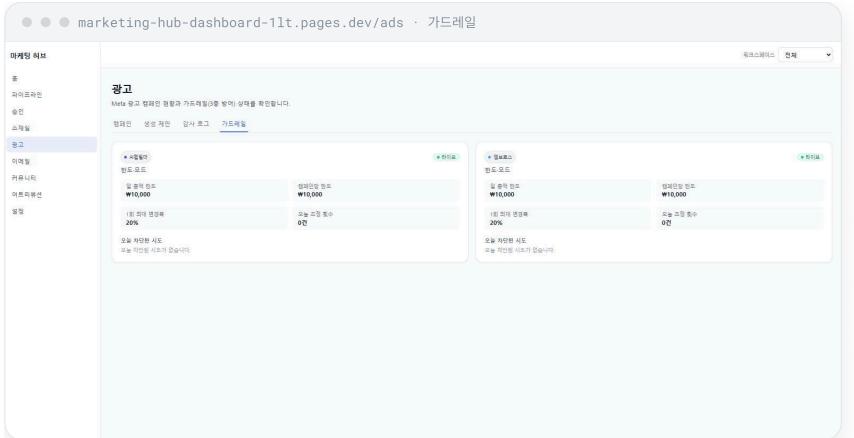
03 / KANBAN

흐름과 이탈의 분리

정상 흐름은 위쪽 가로 트랙, 이탈·보류는 아래쪽 별도 트랙으로 뱌었습니다. 막힌 것이 정상 흐름에 섞여 보이면 무엇이 멈춘는지 한눈에 못 찾기 때문입니다.



상단 초안 → 게시 중 정상 경로
하단 컴플라이언스 실패 · 반려
보류 · 데드레터

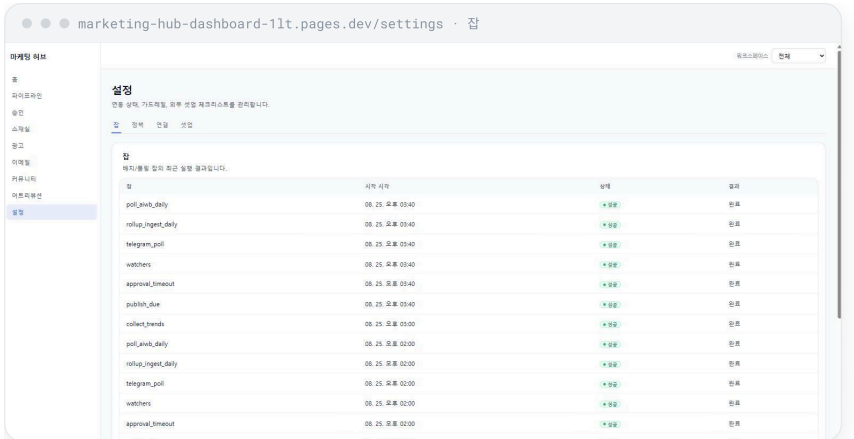


04 / GUARDRAIL

돈이 나가는 경로의 한도

광고는 오작작 하나로 예산이
증발합니다. 그래서 상한을
코드 상수로 두고, 운영 설정은
더 엄격한 방향으로만
반영되게 했습니다.

하드캡 일 30,000원 / 월 300,000원
캠페인 15,000원
변경폭 1회 최대 20%
증액 이 화면에서 불가
텔레그램 승인(T2) 필수

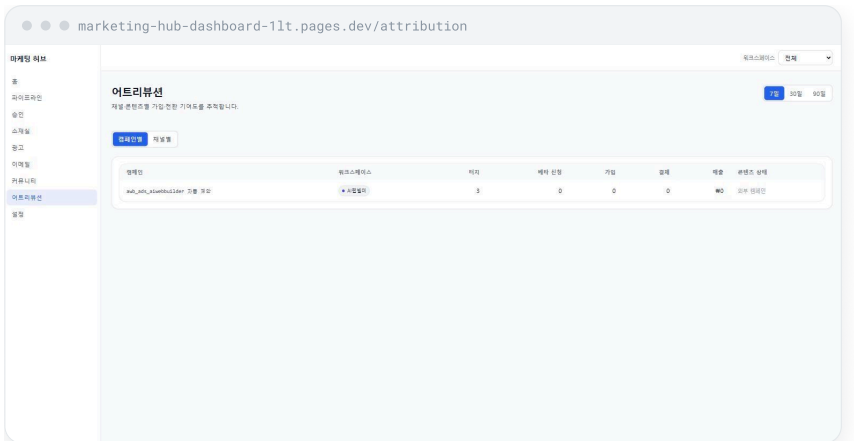


05 / JOBS

배치 실행 이력

17개 잡이 6개 레인에서 돌고,
결과는 전부 job_runs 에
남습니다. 같은 잡이 겹치지
않도록 advisory lock 으로
감싸고, 건별로는
SKIP LOCKED 를 씁니다.

100분 publish_due ·
approval_timeout · watchers
야간 토근 회전 · 밴딧 갱신
미디어 비공개화
기록 성공·실패·소요·오류
(시크릿 제거 후)



06 / ATTRIBUTION

전환 기여도 추적

캠페인 한 줄에 터치 → 베타
신청 → 가입 → 결제 → 매출이
이어집니다. GA4 없이 자체
이벤트로 구현했는데, UTM →
가입 조인은 GA4 로는
불가능하기 때문입니다.

속 캠페인별 / 채널별 전환
연결 프로모코드 발급 시점에
콘텐츠와 바인딩

ENGINEERING

핵심 설계 결정

"돌아가게 만드는 것"과 "혼자 운영해도 안 터지게 만드는 것"은 다릅니다.
아래는 후자를 위해 코드 레벨에 못 박은 불변조건들입니다.

SAFETY

가드레일은 완하 불가

CONSISTENCY

어휘 단일 소스

예산 상한을 guardrails/limits.py 코드 상수로 두고, 런타임 설정은 min(코드상수, 설정) 으로만 반영합니다. DB 를 고쳐도 상한은 내려가지 않습니다.

왜: 운영 설정이 안전장치를 무력화할 수 있으면 그런 안전장치가 아닙니다. 완화하려면 배포가 필요하게 했습니다.

채널·상태·티어·액션 어휘를 models/vocab.py 한 곳에만 정의하고, STATUS_TRANSITIONS 가 상태머신을 강제합니다.

왜: 상태 문자열이 흩어지면 오타 하나가 조용히 흐름을 끊습니다. 어휘를 하나로 묶어 일찍 잡습니다.

IDEMPOTENCY

역등성 3중 방어

- ① advisory lock 으로 잡 중복 실행 차단
- ② 자연키 ON CONFLICT 는 누적이 아닌 대입
- ③ 건별 트랜잭션 + FOR UPDATE SKIP LOCKED

왜: 크론은 재시도되고 겹칩니다. 한 겹만 두면 "같은 글이 두 번 올라가는" 사고가 언젠가 납니다.

TESTABILITY

외부 연동 4모드

Meta 연동은 disabled / stub / dryrun / live 네 모드를 갖고, 상위 계층은 모드를 모릅니다. 토큰이 없어도 전 구간이 완주됩니다.

왜: 외부 API 가 없으면 테스트가 막히는 구조는 결국 테스트를 안 쓰게 만듭니다.

BLAST RADIUS

부작용 초크 포인트 3개

외부로 나가는 모든 행위는 send_email() · apply_ad_action() · publish() 세 함수만 통과합니다. 다른 경로는 금지입니다.

왜: 검증 지점이 흩어지면 언젠가 하나를 빠뜨립니다. 좁은 통로 3개로 모아 여기서만 검사합니다.

SECURITY

오류 정제는 공통 계층

orchestrator/runner.py 의 스크러버가 오류 메시지의 URL 쿼리스트링을 제거한 뒤 기록합니다. 핸들러가 빠뜨려도 시크릿이 남지 않습니다.

왜: 실제로 토큰이 오류 로그에 섞여 나간 적이 있습니다. "각자 조심"이 아니라 통과 지점에서 일괄 처리합니다.

TECH SPEC

개발 스펙

새 스택을 고르는 대신, 이미 운영 중인 두 플랫폼과 같은 인프라를 썼습니다. 혼자 운영하는 시스템에서 스택이 늘어나면 장애 대응 비용이 그만큼 늘기 때문입니다.

BACKEND

- Python 3.11 · FastAPI
단일 앱 · 라우터 21개
- SQLAlchemy 2.0 (async)
테이블 29개 · Alembic
- Pydantic v2
요청·응답 스키마 계약
- Pillow · ffmpeg
카드뉴스 · 릴스 자체 렌더러
- pytest · ruff · mypy
879 통과 · 린트 · 타입체크

FRONTEND

- React 18 · TypeScript
컴포넌트 162 파일
- Vite
빌드 · 개발 서버
- TanStack Query
서버 상태 · 캐시 무효화
- Zustand
클라이언트 상태
- vitest
73건 통과

INFRA & OPS

- Fly.io (도쿄)
scale-to-zero · release_command
- Neon PostgreSQL
싱가포르 · pooled/direct 분리
- Upstash Redis
레이트리밋 · 멍등키
- Cloudflare Pages · R2
대시보드 호스팅 · 미디어
- GitHub Actions cron
내부 API 호출 오케스트레이션

배치 스케줄

레인	주기	잡
every-100m	100분	publish_due · approval_timeout · watchers · telegram_poll · rollup_ingest_daily · poll_aiwb_daily
every-6h	6시간	collect_trends
nightly	매일 03:00 KST	refresh_tokens · poll_embolos_snapshot · bandit_update · media_privatize
morning	매일 05:00 KST	generate_content · adjust_ads
weekly	월 09:00 KST	weekly_report
manual	수동	잡 이름 지정 실행

SCALE

개발 규모

2026년 8월 3일 첫 커밋부터 8월 24일까지 22일간,
설계·구현·배포·운영을 혼자 수행했습니다.

21,160 LOC

백엔드 애플리케이션
Python 129 파일

17,137 LOC

테스트 코드
프로덕션 코드 대비 81%

16,565 LOC

프론트엔드
TypeScript 162 파일

177 커밋

22일간
일 평균 8건

작업 규율

CI 에 || true 를 쓰지 않습니다. 실패는 실패로 남깁니다. 이전 프로젝트에서 CI 가 실패를 무시하도록 설정돼 회귀를 몇 주 동안 숨긴 적이 있었고, 그때부터 "통과한 것처럼 보이는 파이프라인"을 가장 큰 리스크로 봅니다. 코드 리뷰가 "없는 코드"를 잡지 못한다는 것도 같은 경험에서 배웠습니다 — 그래서 명세와 산출물을 별도로 대조하는 단계를 뒀습니다.

강지호

jihono55@gmail.com

화면 캡처는 2026-08-25 운영 환경 실측입니다.

본문 수치(테스트·LOC·커밋)는 저장소에서 직접 계측한 값입니다.

수치를 지어내지 않는 사내 문서 RAG

공모주·IPO 업무 문서(락업 해제일·배정내역·펀드 보유·거래내역)에 자연어로 물으면 **원본 수치**를 **변형 없이 인용**해 답하는 사내 시스템입니다. 금융 문서 QA에서 신뢰의 핵심은 답이 아니라 그 수치가 어느 문서·시트의 어느 행에서 왔는가이므로, 근거 출처를 각주가 아니라 1급 요소로 승격했습니다.

기간	역할	배포	접근
2026.07.08 - 07.16	설계 · 구현 · 배포 단독	Cloud Run · 서울 리전	IAP · Workspace SSO

16,516 줄

Python ·
TypeScript
98개 파일

119 건

테스트 함수
15개 스위트

17,354 청크

BigQuery 색인
OCR 2,262장 포
함

22 개

API 엔드포인트
Flask · SPA 동거

9 일

첫 커밋 → 라이브
25 커밋

OVERVIEW

"자료에 없으면 없다고 답한다"를 제품 규칙으로 만들기

일반 챗봇의 그럴듯한 답변은 금융 실무에서 쓸 수 없습니다. 락업 해제일을 하루라도 틀리면 매도 시점이 어긋나고, 배정 수량을 반올림하면 정산이 깨집니다. 그래서 이 시스템의 설계 목표는 "잘 답하기"가 아니라 "**틀리게 답하지 않기**"였습니다.

PROBLEM

- 공모주 실무 자료가 Google Drive에 **xlsx·pdf·hwp·pptx·이미지**로 흩어져 있고, 같은 수치가 여러 파일에 파생돼 있다
- 엑셀 **병합 셀·다단 헤더**를 일반 파서로 읽으면 값이 행과 어긋나 조용히 오독한다

- "이 수치 어디서 나왔냐"를 되짚을 수 없으면 감사 추적이 끊긴다
- 문서에 비밀번호·계좌번호가 섞여 있어, 색인 자체가 유출 경로가 된다

SOLUTION

- **Layout-Aware 파싱** — 병합 셀 forward-fill, 헤더를 청크마다 반복해 행이 잘리지 않게 한다
- **하이브리드 검색** — 의미(벡터)와 정확 매칭(BM25)을 RRF로 융합. 종목명·펀드명은 BM25가 잡는다
- **근거 강제** — 답변의 모든 수치에 파일·시트·행이 붙고, 자료에 없으면 제공된 자료에서 확인되지 않습니다
- **자격증명 4층 방어** — 색인 제외 · 청크 레dak션 · OCR 캐시 마스킹 · 감사 로그 컬럼 게이트

SIGNATURE · PIPELINE

7단계 파이프라인 — 단계마다 다른 모델을 쓴다

판단 폭이 좁은 단계(재작성·재순위)는 저비용 모델로 고정하고, 생성에만 상위 모델을 씁니다. 임베딩과 벡터 검색은 asia-northeast3(서울)에 두어 고객 금융 데이터가 국내를 벗어나지 않게 했습니다.

01

파싱

확장자별 디스패치. 병합 셀
forward-fill로 값과 행을 붙들고, 수치·단위는 변형하지 않는다.

xlsx · pdf ·
hwp · docx ·
pptx · csv ·
이미지

02

표 인지 청킹

표를 행 중간에서
자르지 않고, 헤더
를 청크마다 반복
해 넣는다. 헤더 없
는 조각은 해석 불
가능한 숫자가 된
다.

MAX_CHUNK_CHARS
2,600

03

중복 제거

사본·구버전이 검
색을 오염시킨다.
MinHash로 근접
중복 파일쌍을 접
고 최신본만 남긴
다.

MinHash 128
perm · eq
0.90

04

임베딩

질의와 문서에 비
대칭 task_type
을 적용하고 병렬
호출한다. 로컬·프
로덕션이 같은 모
델을 공유해 벡터
가 호환된다.

text-
multilingual-
embedding-002
· 768d

05

하이브리드 검색

벡터(의미)와
BM25(종목명·편
드명 정확 매칭)를
RRF로 융합한 뒤
ACL 필터와 Time
Decay를 엮는다.

RRF $k=60$ · λ
0.3 · τ 365일

06

재순위

넓게 회수한 후보
40건을 질의 관련
도로 정밀 재정렬
한다. 저비용 모델
고정 구간.

gemini-3.1-
flash-lite

07

생성·자가검증

수치 총실 프롬프
트로 답을 만들고,
생성된 답의 수치·
날짜·이름을 근거
와 다시 대조해 교
정한다.

gemini-3.5-
flash ·
verify

주입 청크

TOP_K 8

재순위 후보창

40

질의 재작성

N = 3

소스 다양성

그룹당 2

질의 캐시

TTL 3일

운영 비용

< \$20 /
월

SCREENS

근거를 각주가 아니라 1급 요소로

화면당 시그니처는 하나입니다 — 근거 출처 카드. 파일명·시트·행·관련도를 드러내
고, 펼치면 그 수치가 실제로 적혀 있던 원본 행이 그대로 나옵니다. 나머지 UI는 조

용히 물러나 데이터가 주인공이 되게 했습니다.

06rag · 찾기 - 답변과 근거

사내 문서 질의

공모주-IPO 문서에서 근거와 함께 답합니다

찾기 편집-산출

+ 새 대화 시스템

대화 1

+ 새 대화

내 대화 (이 가기)

산일전기 매도가능일이 언제야?
방금 · 질문 1

질 아력 (현재 공부-작성)

4

산일전기 매도가능일이 언제야?
방금

이번 주 작업 해제 등록 알려줘
40일 전

지속운영 수요예측에 참여한 저축...
40일 전

보통드아이에에에스트공모주3호 ...
41일 전

1 현재 대화 전체 삭제

산일전기 매도가능일이 언제야?

답변

산일전기(062040) 의 의무보유확약 물량 매도가능일은 2026-08-14 입니다.

구분	수량	확약기간	매도가능일
기관 의무보유확약	412,500주	1개월	2026-08-14
기관 의무보유확약	168,000주	3개월	2026-10-14
우리사주조합	240,000주	12개월	2027-07-14

상장일은 2026-07-14 이며, 1개월 확약분 412,500주가 8월 14일부터 출회 가능합니다. 3개월 확약분까지 합산하면 총 580,500주가 연내 해제됩니다.

배정내역 원본의 수치를 변형 없이 인용했습니다. 확약기간은 상장일 기준으로 계산됩니다.

근거 4

1 2026년_공모주_배정내역.xlsx 시트: 7행배정 - 행 42 94

2 학업해제일정_2026H2.xlsx 시트: 해제일정 - 행 18 88

3 IPO_수요예측_결과종합.pdf p.23 71

4 펀드별_보유종목_스냅샷.xlsx 시트: 20260731 - 행 156 63

2026년_공모주_배정내역.xlsx 편집 학업해제일정_2026H2.xlsx 편집 IPO_수요예측_결과종합.pdf 편집 펀드별_보유종목_스냅샷.xlsx 편집

공모주-펀드-확약 등 무엇이든 물어보세요 — Enter 전송, Shift+Enter 줄바꿈

원문 보내기

답변 + 근거 출처 카드 — 표는 마크다운으로 렌더링되고, 수치는 `tabular-nums` 로 세로 정렬됩니다. 근거 4 건에는 각각 파일명·시트·행 번호와 관련도 막대가 붙습니다. 하단의 **편집** 칩은 검색에서 편집 탭으로 넘어가는 브리지로, 근거 파일을 그대로 편집 대상에 고정합니다.

근거 카드 펼침 - 원본 행

사내 문서 질의

공모주-IPO 문서에서 근거와 함께 답합니다

찾기 편집-산출

+ 새 대화 시스템

대화 1

+ 새 대화

내 대화 (이 가기)

산일전기 매도가능일이 언제야?
방금 · 질문 1

질 아력 (현재 공부-작성)

4

산일전기 매도가능일이 언제야?
방금

이번 주 작업 해제 등록 알려줘
40일 전

지속운영 수요예측에 참여한 저축...
40일 전

보통드아이에에에스트공모주3호 ...
41일 전

1 현재 대화 전체 삭제

산일전기 매도가능일이 언제야?

답변

산일전기(062040) 의 의무보유확약 물량 매도가능일은 2026-08-14 입니다.

구분	수량	확약기간	매도가능일
기관 의무보유확약	412,500주	1개월	2026-08-14
기관 의무보유확약	168,000주	3개월	2026-10-14
우리사주조합	240,000주	12개월	2027-07-14

상장일은 2026-07-14 이며, 1개월 확약분 412,500주가 8월 14일부터 출회 가능합니다. 3개월 확약분까지 합산하면 총 580,500주가 연내 해제됩니다.

배정내역 원본의 수치를 변형 없이 인용했습니다. 확약기간은 상장일 기준으로 계산됩니다.

근거 4

1 2026년_공모주_배정내역.xlsx 시트: 7행배정 - 행 42 94

2 학업해제일정_2026H2.xlsx 시트: 해제일정 - 행 18 88

3 IPO_수요예측_결과종합.pdf p.23 71

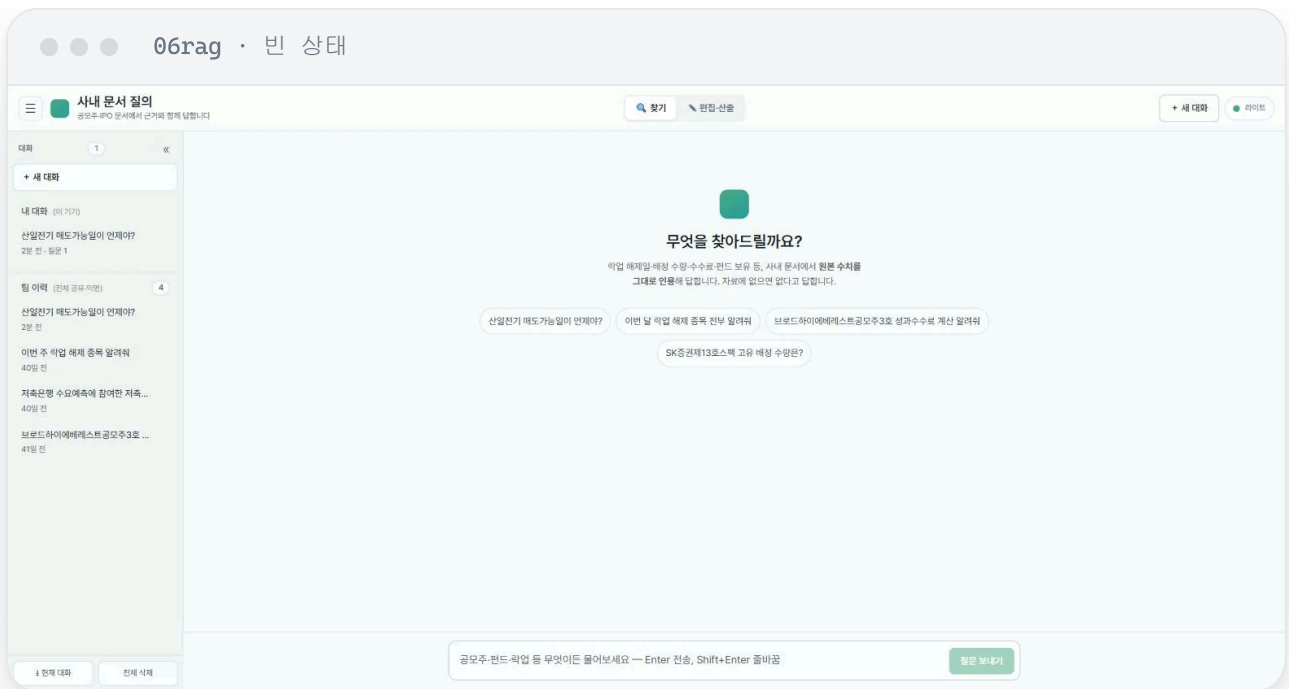
4 펀드별_보유종목_스냅샷.xlsx 시트: 20260731 - 행 156 63

2026년_공모주_배정내역.xlsx 편집 학업해제일정_2026H2.xlsx 편집 IPO_수요예측_결과종합.pdf 편집 펀드별_보유종목_스냅샷.xlsx 편집

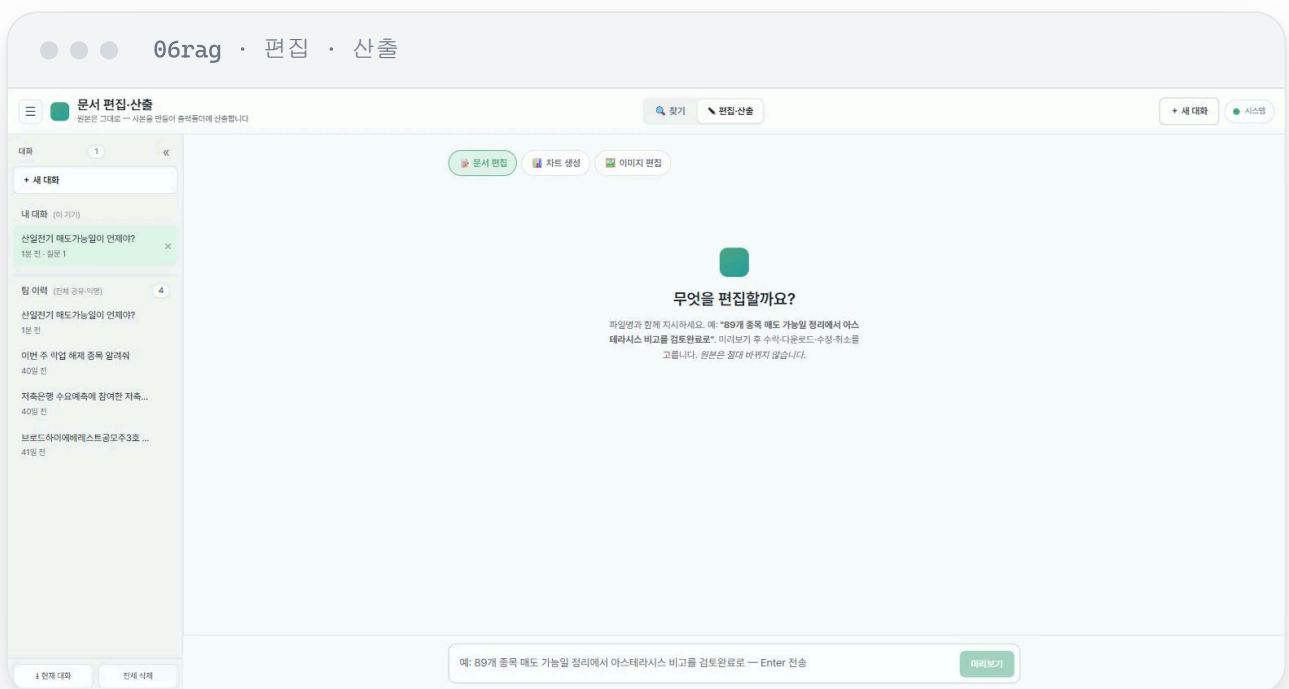
공모주-펀드-확약 등 무엇이든 물어보세요 — Enter 전송, Shift+Enter 줄바꿈

원문 보내기

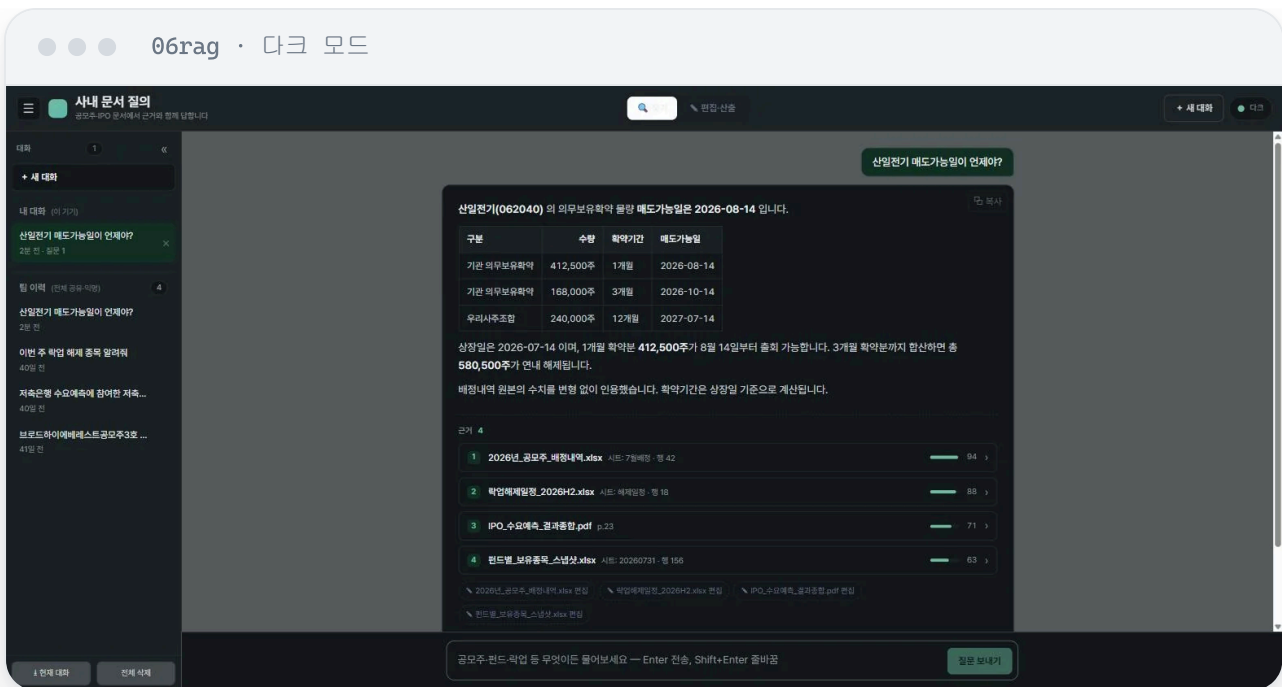
근거 펼침 — 답변이 인용한 그 행이 원문 그대로. 감사 추적이 화면에서 끊기지 않습니다.



빈 상태 — "데이터 없음" 대신 실제 질의 예시로 행동을 유도합니다. 추천 질의는 팀의 인기 정상질의에서 뽑습니다.



편집 · 산출 — 자연어 지시를 셀 좌표로 매핑해 미리보기를 만들고, 수락 시 원본이 아니라 출력폴더에 사본을 만듭니다.



다크 모드 — 라이트의 반전이 아닙니다. elevation을 그림자가 아니라 surface 밝기 단계로 표현하고, 액센트 채도를 20% 낮춰 어두운 배경에서 진동하지 않게 했습니다.

화면에 대한 고지 — 위 캡처는 저장소의 실제 프론트엔드(React 18 · Vite)를 로컬에서 구동해 찍은 것으로, 컴포넌트·CSS·디자인 토큰은 운영 코드 그대로입니다. 다만 백엔드는 사내 GCP 자격증명과 실제 고객 문서가 필요해 이 환경에서 구동할 수 없으므로, **질의 응답 내용과 파일명·수치는 API 계약에 맞춘 시연용 샘플**입니다. 실제 사내 데이터가 아닙니다.

ENGINEERING DECISIONS

증상을 덮지 않고 원인까지 내려간 기록

아래 네 건은 모두 완화책을 먼저 넣어본 뒤, 그것이 증상 가리기임을 확인하고 구조를 바꾼 사례입니다.

01 매일 밤 수백 청크가 재임베딩되는데 아무것도 안 바뀌던 문제

증상

야간 동기화 때마다 **550~830청크**가 재임베딩되고 orphan이 생겼다 사라졌다는 반복했습니다. 문서를 아무도 수정하지 않은 날에도 그랬고, **수렴하지 않았습니다**.

진단

계측 커밋을 먼저 넣어 소스별 집계와 old/new 텍스트 diff를 뽑았습니다. 범인은 한 시트의 **라이브 주가 컬럼**이었습니다 — `GOOGLEFINANCE` 와 웹스크래핑 수식이 매 export마다 재계산되며 값만 바뀌어 **시트 전체 청크의 해시가 같았습니다**. 자동 재계산은 Drive `modifiedTime`을 올리지 않습니다.

조치

`modifiedTime` 게이트를 넣어, 미변경 파일은 재파싱 대신 **BigQuery의 기존 청크를 재사용**하게 했습니다. 재파싱·재임베딩 0. 다만 게이트는 데이터 손실 위험을 만들므로 안전장치를 함께 넣었습니다 — 트립와이어 분모를 활성 청크로 한정하고, orphan 삭제 스코프를 활성 파일로 좁히고, **동결된 파일에 자격증명 컬럼이 있으면 강제로 재파싱**해 보안 가드가 자가치유되게 했습니다.

02 동의어를 아무리 늘려도 원천 문서가 안 잡히던 검색

증상

"홀딩" 관련 질의를 하면 파생 문서만 걸리고, 그 수치가 실제로 계산된 **원천 문서(배정내역·명세부)**가 **top-K**에 들어오지 않았습니다.

1차 완화와 그 한계

도메인 동의어군(홀딩 ↔ 배정내역 ↔ 명세부)을 기본값에 넣어 급한 불은 꺾었습니다. 하지만 이걸 **여휘를 손으로 이어 붙인 것**이라, 새 파생 관계가 생길 때마다 사람이 등록해야 합니다. 검색이 문서 사이의 관계를 모른다는 사실은 그대로였습니다.

근본 해결

xlsx 수식에서 **파생 관계를 추출**했습니다. 크로스시트 참조, 크로스파일 [1] 참조, IMPORTRANGE를 파싱해 (참조하는 문서 → 참조되는 문서) 엮지를 인제스트 시점에 사전계산하고, 검색 시 상위 소스의 원천 문서를 **재순위 후보창의 꼬리에 주입**합니다. 후보 수는 그대로라 **재순위 지연 증가는 0**이고, 킬 스위치를 끄면 완전 no-op입니다.

03 색인 자체가 유출 경로가 되는 문제

위험

업무 문서에는 비밀번호·보안카드·계좌번호가 섞여 있습니다. 이미지 **2,262장을 일괄 OCR**하기로 한 시점에, 텍스트화된 자격증명이 여러 저장 계층에 평문으로 남게 된다는 게 드러났습니다.

계층별 갭

최종 청크에는 레dak션이 걸려 있었지만 **OCR 캐시**에는 약한 마스킹만 적용돼 이미지 안의 비밀번호가 BigQuery에 그대로 상주했습니다. 편집 감사 로그도 값 단위 레dak션만으로는 **컬럼 전체가 자격증명**인 경우를 잡지 못했습니다.

4층 방어

① 자격증명 파일은 **색인 자체를 제외**, ② 청크 텍스트 레dak션, ③ OCR 캐시도 같은 레dak션으로 승격, ④ 감사 로그는 값이 아니라 **컬럼명으로 게이트**. 채팅 스크린샷 다수는 OCR 후 내용 기준으로 제외했고, 배포 전 BigQuery를 감사해 **자격증명 컬럼 잔존 0/14,662**를 확인했습니다.

04 모델을 바꿨을 때 과거 정답이 깨졌는지 어떻게 아는가

문제

모델·프롬프트·검색 파라미터를 바꾸면 어제까지 맞던 답이 조용히 틀려질 수 있습니다. RAG는 **회귀**가 눈에 안 보이는 시스템입니다.

LLM JUDGE를 안 쓴 이유

채점을 LLM에 맡기면 판정 자체가 흔들려 **테스트가 flaky해집니다**. 대신 생성 규칙 중 결정적으로 검사 가능한 것만 골랐습니다 — "날짜는 원본 그대로"는 정규식으로 확인할 수 있습니다.

골든셋 회귀 하네스

운영 질의 로그에서 사람이 검증한 정답을 **골든셋으로 동결**하고, 변경 후 파이프라인을 재실행해 핵심 날짜·수치가 보존되는지 검사합니다. **날짜 커버리지 100%가 하드 게이트**, 수치는 90%. 테이블은 append-only로 두어 스트리밍 버퍼 DML 제약을 피했고, 하네스는 Cloud Run Job으로 분리해 스케줄 실행합니다.

INFRASTRUCTURE

키 없는 운영, 서울에 머무는 데이터

서비스 계정 키 파일을 만들지 않았습니다. 프로덕션 자격은 Cloud Run 런타임 서비스 계정이고, SMTP 같은 외부 자격만 Secret Manager에서 런타임에 가져옵니다 — 코드·이미지·환경변수 어디에도 비밀이 없습니다.

SERVING

API + SPA

Cloud Run · rag-api

컨테이너

멀티스테이지 (node → python)

인증

IAP · Workspace SSO

질의자 신원

X-Goog-Authenticated-User-Email

DATA

벡터 검색

BigQuery VECTOR_SEARCH

로컬 동형 저장소

NumPy · ChromaDB

파생 테이블

lookup · lineage · cards

운영 로그

query_log · feedback · edit_log

JOBS

야간 동기화

rag-sync · 03:00 KST

회귀 평가

rag-eval

락업 D-day 알림

rag-lockup-alert · D-7 · D-1

스케줄

Cloud Scheduler

TECH SPEC

개발 스펙

백엔드

Python 3.11

Flask

google-cloud-bigquery

google-api-python-client

rank_bm25

datasketch (MinHash)

openpyxl

pdfplumber

python-docx · pptx

Pillow

AI · 검색

Vertex AI

gemini-3.5-flash

gemini-3.1-flash-lite

text-multilingual-embedding-002 (768d)

BigQuery VECTOR_SEARCH

BM25 + RRF

Time Decay

LLM 재순위

질의 재작성

자가검증

프론트엔드

React 18

TypeScript

Vite 6

react-markdown · remark-gfm

OKLCH 디자인 토큰

다크 모드 별도 팔레트

WCAG AA

인프라

Cloud Run

BigQuery

Cloud Run Jobs

Cloud Scheduler

Secret Manager

IAP

Docker 멀티스테이지

키리스 서비스 계정

asia-northeast3

연동

Google Drive API

Google Sheets API

도메인 전체 위임

지수 백오프 + 커서

SMTP 알림

입력 포맷

xlsx · xls

pdf

hwp · hwpX

docx · pptx

csv

이미지 8종 (Gemini 멀티모달 OCR)

텔레그램 · 카카오톡 export

품질 · 보안

pytest 119건

골든셋 회귀 하네스

ACL 질의 필터

자격증명 4층 레dak션

PII 마스킹

편집 감사 로그

원본 불변 편집

SCALE

개발 규모

2026년 7월 8일 첫 커밋부터 7월 16일까지 9일간, 설계·구현·배포를 혼자 수행했습니다.

9,896 줄

백엔드

Python 51
개 파일 · src
모듈 44개

3,230 줄

프론트엔드

React ·
TypeScript
19개 파일

2,594 줄

테스트

함수 119개 ·
15개 스위트

442 줄

인프라 · 배
포

배포 스크립
트 7개 · 스키
마 ·
Dockerfile

25 커밋

9일간

일 평균 2.8
건

본문 수치는 저장소에서 직접 계측한 값입니다. 코드 규모는 소스 집계(생성 파일 `package-lock.json` 제외), 파이프라인 파라미터와 모델명은 `config.pycloud/deploy.md` 기준입니다 — 저장소 `README.md`에는 이전 세대 모델명이 남아 있어, 코드를 진실의 원천으로 삼았습니다. 색인 청크 수와 OCR 처리량은 운영 로그에 기록된 실측치입니다. 저장소는 사내 자료를 다루므로 `private`이며, 코드 열람이 필요하시면 이메일로 요청해 주세요.

• Product Engineer · 2026

제품을 만들고, 그 제품을 검증하는 도구까지 만듭니다.

기획·설계·구현·운영을 혼자 끝내는 것을 기준으로 삼습니다. 아래 두 프로젝트는 모두 실제로 돌아가고 실제 산출물을 냅니다 — 이 문서의 화면과 리포트는 전부 실행 결과를 그대로 캡처한 것입니다.

[github.com/StockHedge](#) [jihono55@gmail.com](#) 최근 업데이트 2026.08

2개

단독 개발·운영 제품

10,232줄

TypeScript · Python

60커밋

두 저장소 추적

47회

실제 테스트 세션 실행

20명

설계한 평가 페르소나

PROJECT 01

AI 베타테스터

출시 전에, 사람 대신 사람처럼 써보는 QA 에이전트. 게임과 웹 서비스를 페르소나 20명의 시선으로 직접 플레이·탐색하고, 근거가 붙은 채점 리포트를 만든다.

Next.js 16

React 19

TypeScript

Tailwind CSS 4

Supabase · Postgres · RLS

Anthropic SDK

Playwright

Zod

Android adb

Docker

npm workspaces

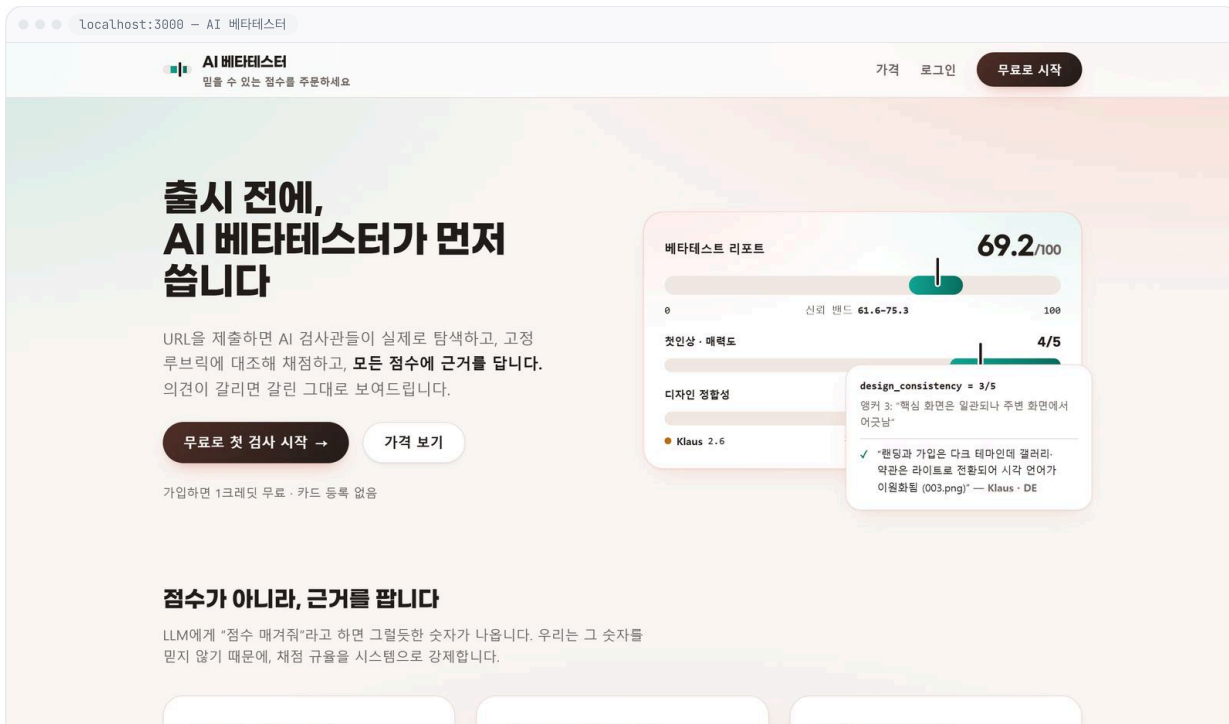
Vercel

PROBLEM

출시 직전 QA는 비싸고 느리다. 자동화 테스트는 "버튼이 눌리는가"는 잡아내지만 "재미가 없다 / 어디를 눌러야 할지 모르겠다"는 잡지 못한다. 그렇다고 실제 베타테스터를 모으면 모집·일정·비용이 전부 걸린다. 결국 사람의 판단이 필요한 영역이 통째로 비어 있는 채로 출시하게 된다.

SOLUTION

나이·국가·성향이 다른 페르소나가 에뮬레이터와 브라우저에서 제품을 직접 조작하고, 축별 앵커 루브릭으로 채점한다. 모든 점수에 관찰 근거를 강제해 "왜 3점인가"가 스크린샷 단위로 추적된다. 결과는 외부 리소스 의존이 0인 자립형 HTML 리포트 한 장이다.



N

모든 축 점수에 실제 관찰(스크린샷-URL-콘솔 오류) 인용이 강제됩니다. 리프트의 전소 연애는 것과같이 실제 한 마디

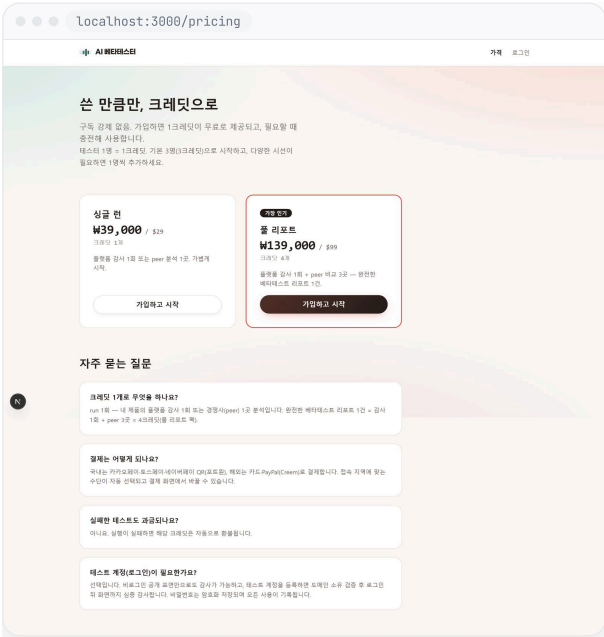
점수는 점이 A와 범위

검사관 여러 명의 판단이 같려면 밴드가 넓어지고, 어느 쪽으로 갈렸는지까지 보여줍니다. 만데는 파다아 포이지 오차가

관찰 못 한 축은 N/A

도달하지 못한 화면은 추측으로 채점하지 않고 N/A로 제외한 뒤 가중치를 재분배합니다. 초점이 어떤 커브리지에서

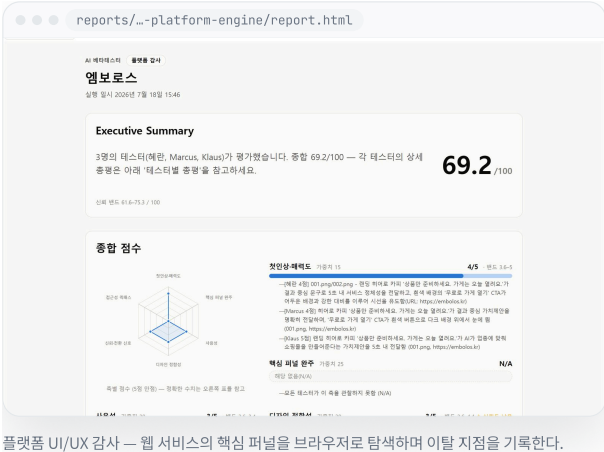
서비스 랜딩 — router.js 16 App Router · React 19 서버 컴포넌트, Supabase 세션 유무를 서버에서 판정해 랜딩과 대시보드로 분기한다.



요금제 — 구독형(로컬 러너)과 종량형(API 엔진), 원가 구조가 다른 두 실행 경로를 각각 상품화했다.



게임 플레이테스트 리포트 — 종합 69.4/100, 신뢰 밴드 59.6–79.2. 각 축 아래에 관찰 근거와 스크린샷 파일명이 붙는다.



플랫폼 UI/UX 감사 — 웹 서비스의 핵심 파일을 브라우저로 탐색하며 이탈 지점을 기록한다.



경쟁사 비교 — 공개 표면만 같은 루브릭으로 훑어 비교 매트릭스를 만든다. 계정 생성·품 제출은 하지 않는다.

설계에서 가장 신경 쓴 것

01

근거 없는 점수를 만들지 않는다

LLM 채점의 최대 리스크는 그럴듯한 숫자를 지어내는 것이다. 채점 규율을 별도 모듈로 분리해 관찰 인용이 없는 축은 채점 자체를 하지 않도록 강제했다. 리프트의 모든 점수에는 근거 문장과 012.png 같은 스크린샷 참조가 따라붙는다.

03

비용 경로를 돌로 나눴다

API 총량 과금은 세션당 실측 \$0.11–0.16. 여기에 더해 사용자의 기존 구독을 그대로 쓰 는 로컬 러너 경로를 별도로 만들어, 원가 구조가 다른 두 상품을 같은 엔진 위에 올렸다.

02

단일 점수 대신 신뢰 밴드

"69.4점"만 보여주면 정말해 보이는 착시가 생긴다. 세션 간 의견 분산을 59.6–79.2 구간으로 함께 노출해 얼마나 확실할 수 있는 점수인지까지 읽히게 했다. 재실행 시 이전 런과의 밴드 diff도 산출한다.

04

크롤링 유리를 코드로 강제

경쟁사 분석은 경계가 분명해야 한다. robots.ts로 robots.txt를 준수하고, peer 분석 에이전트는 계정 생성·품 제출 없이 공개 표면만 관찰하도록 권한 자체를 제한했다.

출어진 개발 커뮤니티의 최근 72시간을 3일마다 한국어 리포트로 묶어 PDF로 만들고 카카오톡으로 보내는 무인 파이프라인.

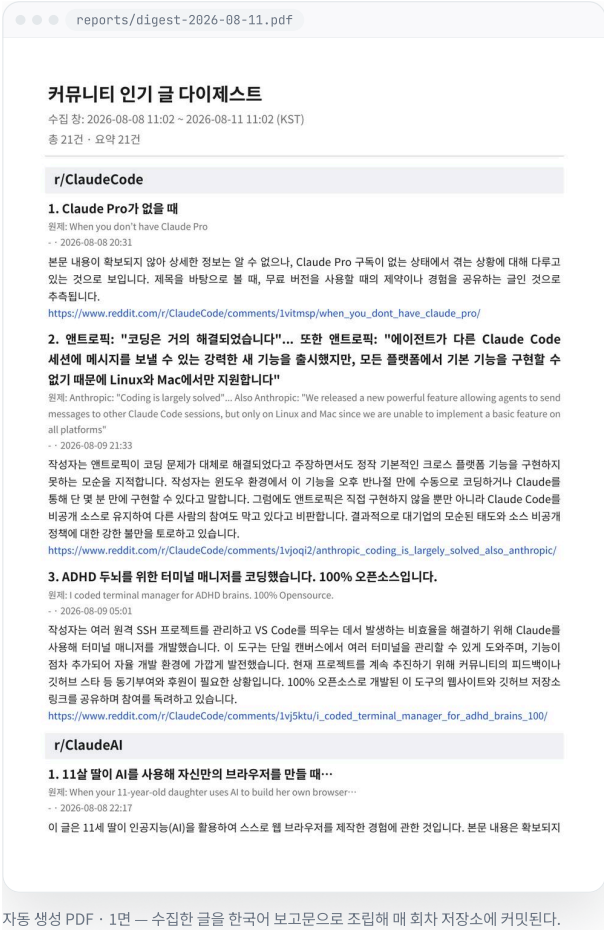
- Python 3.11
- GitHub Actions
- Reddit API
- HN Algolia
- RSS
- Kakao OAuth
- Google Drive API
- fpdf2
- fontTools 서버셋

PROBLEM

바이브 코딩 · Claude Code 관련 정보가 Reddit, Hacker News, 깃뉴스에 흩어져 있다. 매일 다섯 군데를 돌아 훑는 습관은 현실적으로 유지되지 않고, 한 번 놓치면 다시 따라잡을 방법이 없다.

SOLUTION

수집 → 한국어 보고문 조립 → PDF 생성 → Drive 공개 업로드 → 카카오톡 발송을 하나의 무인 루틴으로 묶었다. GitHub Actions에서 돌기 때문에 내 PC가 꺼져 있어도 3일마다 도착한다.



자동 생성 PDF · 1면 — 수집한 글을 한국어 보고문으로 조립해 해 최적 저장소에 컷밋된다.

· 2면 — 한글 표시를 위해 Noto Sans KR를 fontTools로 서버셋해 PDF에 임베드했다.

주기 스케줄링의 함정을 코드로 우회한 부분

cron의 */3은 "3일마다"가 아니다. day-of-month 기반이라 31일 달을 지나면 주기가 어긋난다. 그래서 매일 깨우되 epoch-day를 3으로 나눈 나머지로 실행 여부를 결정하는 게이트를 만들어, 월 경계와 무관하게 정확한 3일 간격을 유지했다.

```
# cron은 매일 00:00 UTC(=09:00 KST)에 깨우고,
# epoch-day % 3 게이트로 3일 주기를 만든다.
elif [ $(($(date +%s) / 86400 % 3)) -eq 0 ]; then
echo "run=true" >> "$GITHUB_OUTPUT"
```

설계에서 가장 신경 쓴 것

- 01

의존성을 1개로 묶었다

외부 패키지는 PDF 생성용 fpdf2 하나뿐이고 나머지는 전부 표준 라이브러리다. 무인 루틴에서 의존성은 곧 미래의 장애 지점이라, 편의보다 수명을 택했다.
- 02

커뮤니티 단위 실패 격리

수집 대상 하나가 죽어도 다이제스트는 나가야 한다. 수집기를 커뮤니티 단위로 격리해 한 곳의 장애가 전체 발송을 막지 못하게 했다. Reddit 레이트리미트는 잔여량을 감지해 백오프한다.
- 03

토큰이 만료돼도 멈추지 않게

카카오 refresh token 만료(invalid_grant)는 무인 파이프라인이 조용히 죽는 대표 원인이다. 새 토큰 수신 시 GitHub Secret을 자동 갱신하고, 자동 갱신이 불가하면 재인증 안내를 카카오톡 발송해 사람이 알아채게 만들었다.
- 04

수집 대상을 리서치로 확정

넣을 곳뿐 아니라 제외할 곳과 그 사유까지 문서로 남겼다 — robots 차단, 로그인 벽, 무료 API 부재, ToS상 스크래핑 금지. 나중에 "왜 여긴 없지?"를 다시 조사하지 않기 위한 기록이다.

다루는 것들

두 프로젝트에서 실제로 설계 · 구현 · 운영까지 해본 범위만 적었습니다.

프론트엔드	TypeScript · React 19 · Next.js 16(App Router · 서버 컴포넌트) · Tailwind CSS 4 · WCAG AA 기준 접근성 구현
백엔드 · 데이터	Supabase(Postgres · Auth · RLS) · 마이그레이션 관리 · 서버 전용 키 격리 · Python 3.11
자동화 · 인프라	GitHub Actions(스케줄 · 시크릿 회전) · Docker · Vercel · npm workspaces 모노레포
테스트 · 계속	Playwright(브라우저 자동화 · E2E) · Android adb 기기 제어 · 자체 채점 엔진 설계
AI 응용	Anthropic SDK 기반 에이전트 파이프라인 · 페르소나/루브릭 설계 · 근거 강제 채점 · 실측 기반 원가(COGS) 설계
연동	OAuth 2.0(카카오 · Google) · refresh token 회전 · Reddit / HN Algolia / RSS 수집 · robots.txt 준수

함께 만들 것이 있다면

제품 설계부터 운영까지 혼자 굴러본 경험을 팀에서 쓰고 싶습니다.

저장소는 모두 공개되어 있으니 코드부터 보셔도 좋습니다.

이메일 보내기 — jihono55@gmail.com

GitHub — github.com/StockHedge